

TCGS 项目技术架构报告

目录

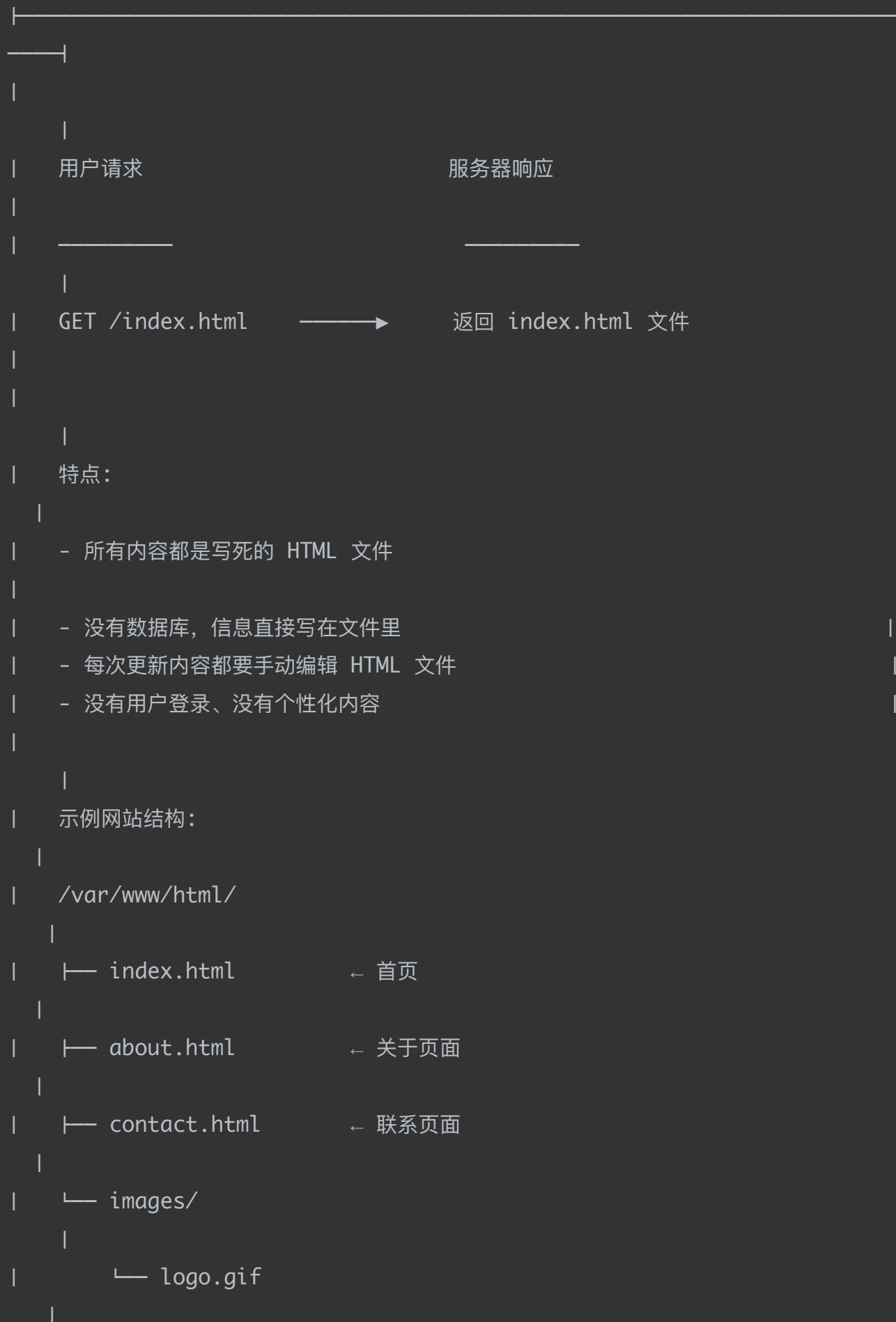
1. 技术演进：从石器时代到现代Web
2. 项目概述与技术栈总览
3. 数据存储演进：SQL、PostgreSQL 与关系型数据库
4. 后端框架之争：FastAPI vs Java Spring Boot
5. 数据库迁移：Alembic 的前世今生
6. 前端架构：Vue 3 与 Pinia 状态管理
7. 容器化革命：Docker 如何解决依赖地狱
8. 反向代理：为什么需要 Nginx
9. 对象存储：MinIO 是什么
10. 系统架构图与请求流转
11. Vue 响应式原理
12. 4+1 视图模型

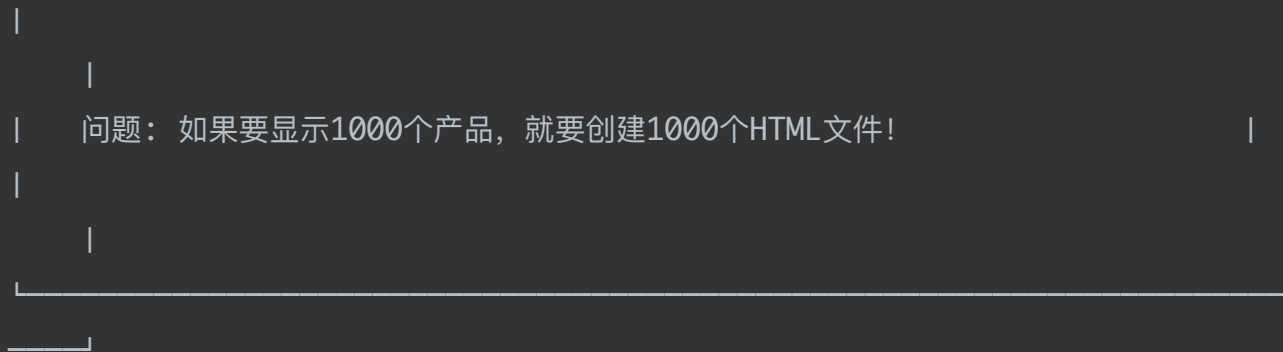
1. 技术演进：从石器时代到现代Web

要理解现代 Web 技术栈的设计，我们需要先回顾一下 Web 开发的演进历程。这不仅能帮助我们理解"为什么需要这些技术"，还能让我们明白它们各自解决了什么问题。

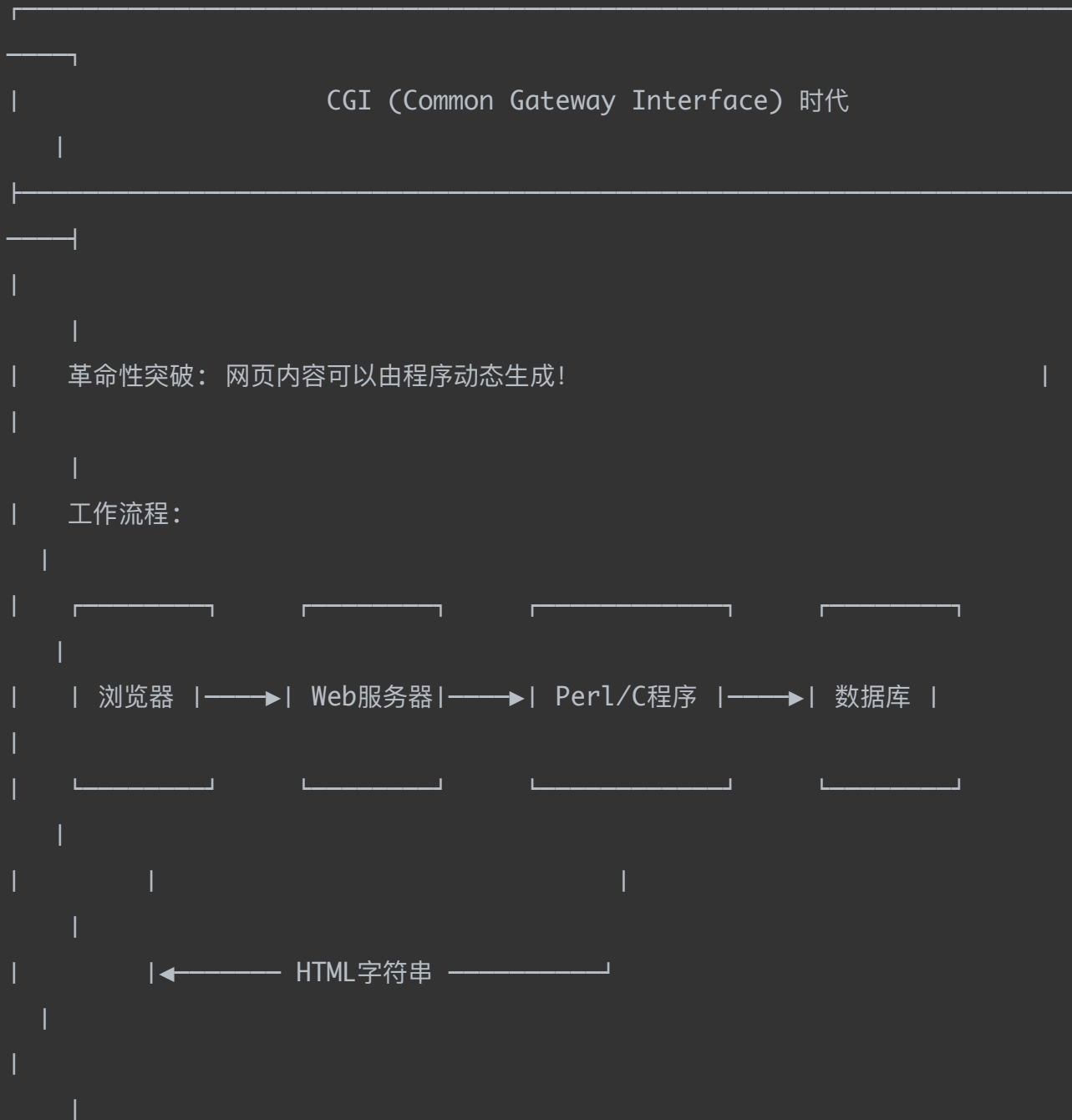
1.1 石器时代：静态 HTML (1990s 初期)

1990年代初期：静态网页时代





1.2 青铜时代：CGI 动态网页 (1990s 中后期)



```
|  示例 Perl CGI 脚本：
|
|
|
|-----|
|  |  #!/usr/bin/perl
|  |
|  |  print "Content-type: text/html\n\n";
|  |
|  |  print "<html><body>";
|  |
|  |  print "当前时间: " . localtime();
|  |
|  |  print "</body></html>";
|  |
|
```

问题：

- 每个请求都要启动一个新进程，性能极差
- 代码和 HTML 混在一起，难以维护
- 安全漏洞多（SQL注入、命令注入）
- 没有会话管理，每个请求都是独立的

1.3 铁器时代：PHP/JSP/ASP 与 MVC 框架 (2000s)

服务端渲染 + MVC 架构时代


```
| | Topic.java      TopicController  topic_detail.jsp |
|
| |
| |
|
|
|-----|
|
| |
| Java Servlet 示例 (2000年代风格):
|
|
|-----|
| | @WebServlet("/users")
| |
| | public class UserServlet extends HttpServlet {
| |
| |     protected void doGet(HttpServletRequest req,
| |
| |                             HttpServletResponse resp) {
| |
| |         List<User> users = userDao.findAll();
| |
| |         req.setAttribute("users", users);
| |
| |         req.getRequestDispatcher("/users.jsp")
| |
| |             .forward(req, resp);
| |
| |     }
| |
| | }
|
|-----|
|
| |
```

问题：

- 服务端渲染，每次操作都要刷新整个页面
- 前后端代码耦合严重
- 部署复杂（Tomcat/WebLogic 配置繁琐）
- 扩展困难，单体应用越来越臃肿

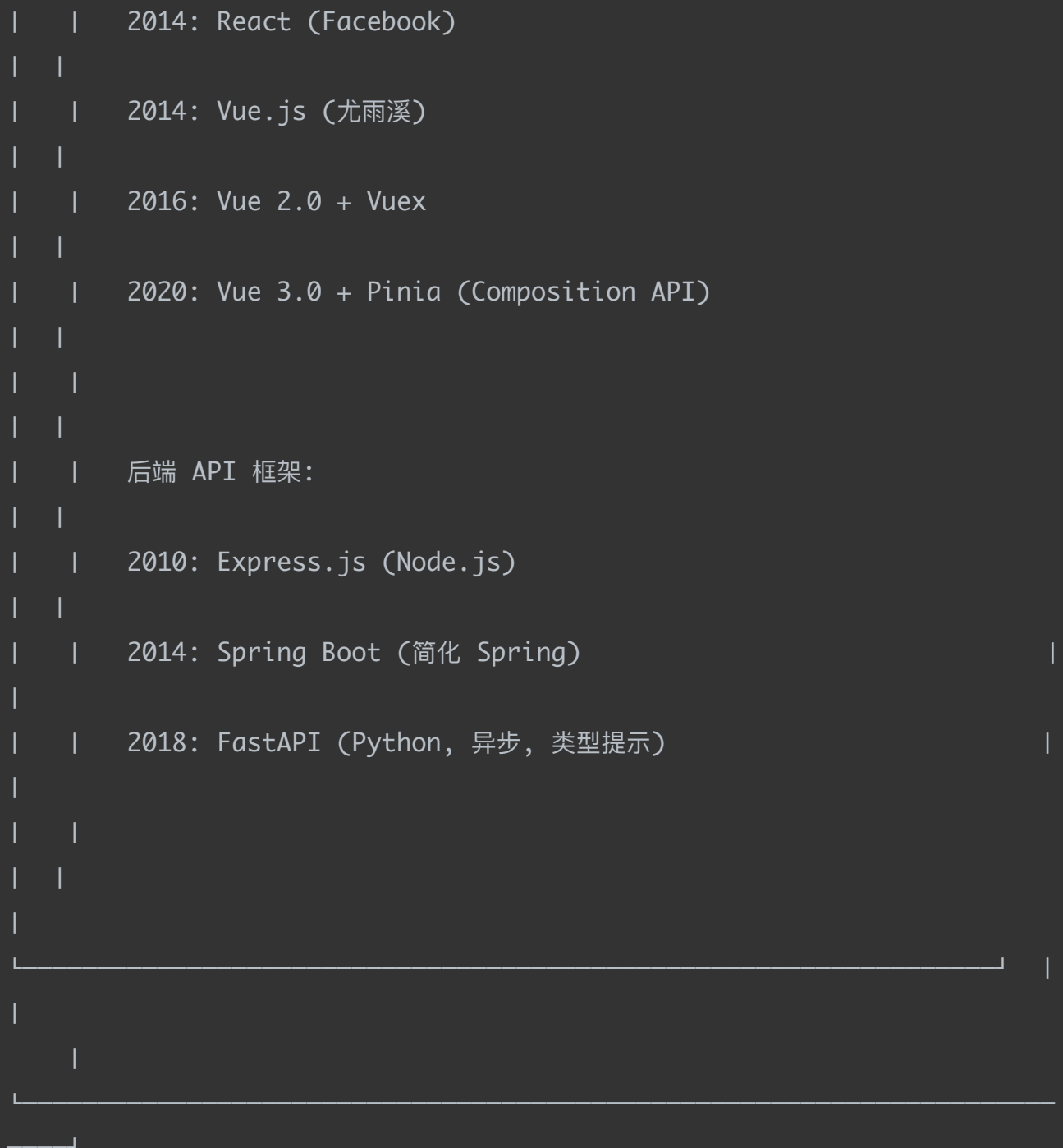
1.4 工业时代：前后端分离 + SPA (2010s)

前后端分离 + 单页应用（SPA）时代

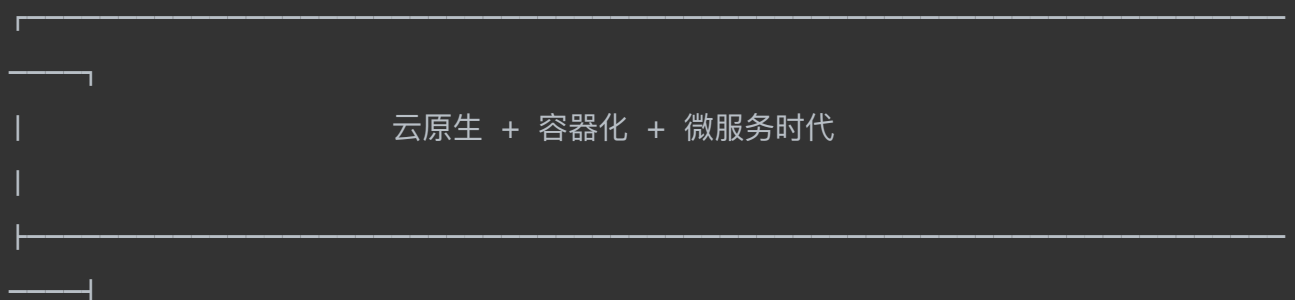
革命性变化：

1. 前端不再只是"模板"，而是独立的应用
2. 后端只提供数据 API（JSON），不管页面渲染
3. AJAX/Fetch 实现局部刷新，用户体验大幅提升
4. 前后端可以独立开发、独立部署

架构对比：



1.5 现代：云原生与容器化 (2020s)



现代 Web 应用面临的挑战：

- 1. "在我机器上能跑" → 需要环境一致性 → Docker 容器
- 2. 手动部署易出错 → 需要自动化 → CI/CD + Docker Compose
- 3. 数据库结构变更混乱 → 需要版本控制 → Alembic/Flyway
- 4. 前端状态管理复杂 → 需要状态容器 → Pinia/Redux
- 5. API 文档维护困难 → 需要自动生成 → FastAPI/Swagger
- 6. 文件存储分散 → 需要统一管理 → MinIO/S3
- 7. 跨域、负载均衡 → 需要反向代理 → Nginx

TCGS 项目采用的现代技术栈：

问题	解决方案
前端 UI	Vue 3 + TypeScript
前端状态管理	Pinia
后端 API	FastAPI + Pydantic
数据持久化	PostgreSQL + SQLAlchemy
数据库版本控制	Alembic

- FastAPI: Python 生态丰富, AI/算法团队熟悉

- PostgreSQL: 开源免费, 功能强大, 生产级稳定

2. 开发效率与运行效率的平衡

- Python 写起来快, FastAPI 运行性能接近 Go

- TypeScript 有类型检查, 减少运行时错误

- Docker 一次配置, 处处运行

3. 团队技能与技术匹配

- 算法团队熟悉 Python → FastAPI

- 前端工程师熟悉 Vue → Vue 3

- 运维熟悉 Docker → 容器化部署

4. 可维护性与可扩展性

- Alembic 管理数据库变更, 可追溯可回滚

- Pinia 管理前端状态, 逻辑清晰

```
|      - Docker Compose 定义服务依赖，一键部署
|
|
|
|      |
|_____|
_____
```

2. 项目概述与技术栈总览

2.1 项目简介

TCGS (Topic & Capability Governance System) 是一个算法能力管理平台，用于组织研发工作流程。核心功能包括：

- **课题管理 (Topics):** 管理研究课题的完整生命周期
- **阶段管理 (Stages):** workflow 阶段推进和交付物追踪
- **产能管理 (Capacity):** 人员和资源分配
- **知识库 (Wiki):** 文档和知识沉淀
- **洞察分析 (Insights):** KPI 统计和分析

2.2 技术栈总览

```
|_____
|
|      技术栈架构      |
|_____
|
|
|
|
|      |_____
|
|      前端 (Frontend)  |
|
|
```

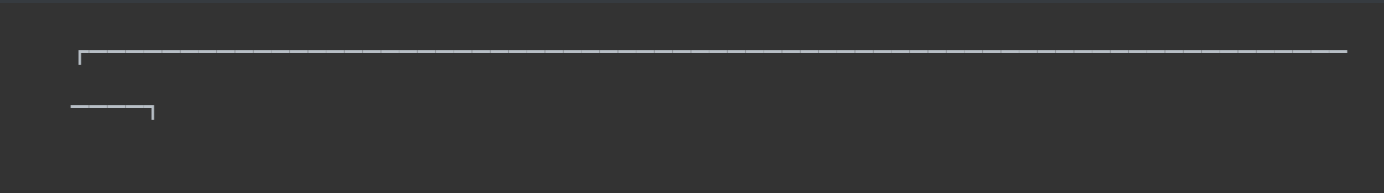

技术组件清单

层级	技术	版本	用途
前端框架	Vue.js	3.5.13	响应式 UI 框架
类型系统	TypeScript	5.7.2	静态类型检查
状态管理	Pinia	3.0.4	全局状态管理
路由	Vue Router	4.x	SPA 路由
UI 组件库	Element Plus	2.13.1	企业级 UI 组件
CSS 框架	TailwindCSS	4.1.10	原子化 CSS
构建工具	Vite	6.4.1	快速开发服务器和构建
HTTP 客户端	Axios	1.13.2	API 请求
后端框架	FastAPI	0.115.0	异步 Python Web 框架
ORM	SQLAlchemy	2.0.35	对象关系映射
数据验证	Pydantic	2.9.2	数据校验和序列化
数据库迁移	Alembic	1.13.2	数据库版本控制
数据库	PostgreSQL	15	关系型数据库
文件存储	MinIO	latest	S3 兼容对象存储
Web 服务器	Nginx	Alpine	反向代理和静态文件服务
容器化	Docker	Compose	应用容器化部署

3. 数据存储演进：SQL、PostgreSQL 与关系型数据库

很多初学者会问：**SQL 和 PostgreSQL 是什么关系？MySQL、SQLite 又是什么？** 这一章我们来彻底搞清楚这些概念。

3.1 SQL 是一种语言，不是数据库



核心概念澄清

SQL (Structured Query Language) = 结构化查询语言

- SQL 是一种"语言", 就像 Python、Java 是编程语言一样
- SQL 专门用来操作数据库: 查询、插入、更新、删除数据
- SQL 本身不存储数据, 它只是告诉数据库"要做什么"

类比:

SQL 语言 ↔ 中文/英文

数据库系统 ↔ 使用这种语言的国家/地区

你说"请给我一杯水"(SQL语句)

不同的服务员(数据库)都能理解, 但可能有口音差异(方言/扩展)

常见的关系型数据库系统 (都使用 SQL 语言):

数据库系统		特点
PostgreSQL	功能最强大，完全开源，企业级	
MySQL	最流行，Oracle 旗下，Web 应用常用	
SQLite	轻量级，单文件，嵌入式应用	
Oracle	商业数据库，银行/电信等大企业使用	
SQL Server	微软产品，Windows 生态	

3.2 SQL 语言基础示例

```
-- SQL 是所有关系型数据库通用的语言
-- 以下语句在 PostgreSQL、MySQL、SQLite 中都能运行

-- 1. 创建表（定义数据结构）
CREATE TABLE users (
    id SERIAL PRIMARY KEY,          -- 自增主键
    email VARCHAR(255) NOT NULL,    -- 邮箱，不能为空
    name VARCHAR(100),              -- 姓名
    created_at TIMESTAMP DEFAULT NOW() -- 创建时间，默认当前时间
);

-- 2. 插入数据
INSERT INTO users (email, name) VALUES ('zhang@example.com', '张三');
INSERT INTO users (email, name) VALUES ('li@example.com', '李四');
```

```

-- 3. 查询数据
SELECT * FROM users;                -- 查询所有用户
SELECT name, email FROM users WHERE id = 1;    -- 查询 id=1 的用户
SELECT * FROM users ORDER BY created_at DESC;  -- 按创建时间倒序

-- 4. 更新数据
UPDATE users SET name = '张三丰' WHERE id = 1;

-- 5. 删除数据
DELETE FROM users WHERE id = 2;

-- 6. 关联查询 (SQL 的精髓)
SELECT topics.title, users.name as author
FROM topics
JOIN users ON topics.user_id = users.id
WHERE topics.status = 'OPEN';

```

3.3 为什么选择 PostgreSQL?

PostgreSQL vs MySQL 详细对比			
特性对比：			
特性	PostgreSQL	MySQL	
JSON 支持	原生 JSONB, 可索引	JSON 类型, 较弱	

	全文搜索	内置，功能强大	需要插件		
	事务隔离	完整 ACID	取决于存储引擎		
	扩展性	可编写自定义函数	有限		
	标准符合度	最接近 SQL 标准	有较多非标准扩展		
	复杂查询	窗口函数、CTE 强大	8.0 后才完善		
	开源协议	PostgreSQL License	GPL (Oracle 控制)		

TCGS 项目选择 PostgreSQL 的理由：

1. JSONB 支持：课题的阶段配置、成功标准等都是 JSON 格式

`stages = Column(JSONB)` # 可以直接存储复杂的嵌套结构

2. 全文搜索：Wiki 知识库需要搜索功能，PostgreSQL 内置支持

`SELECT * FROM wiki_pages`

`WHERE to_tsvector('chinese', content) @@ to_tsquery('算法');`

3. 完全开源：不受商业公司控制，社区活跃

4. 云服务支持：AWS RDS、阿里云、腾讯云都提供托管服务

3.4 ORM：让代码操作数据库更优雅

什么是 ORM?

ORM = Object-Relational Mapping (对象关系映射)

问题：直接写 SQL 有什么不好?

传统方式：手写 SQL (容易出错，难以维护)

```
cursor.execute("""
```

```
    INSERT INTO users (email, name, created_at)
```

```
    VALUES (%s, %s, %s)
```

```
""", (email, name, datetime.now()))
```

问题：SQL 注入风险、字符串拼接繁琐、IDE 无法检查

ORM 方式：用 Python 对象操作

```
user = User(email=email, name=name)
```

```

| db.add(user)
|
| db.commit()
|
| # 优势：类型安全、IDE 补全、自动防 SQL 注入
|
|
|_____
_____

```

TCGS 项目中的 SQLAlchemy ORM 示例:

```

# backend/app/models/user.py
from sqlalchemy import Column, Integer, String, DateTime, Enum
from sqlalchemy.orm import relationship
from datetime import datetime

class User(Base):
    """用户模型 - 对应数据库中的 users 表"""
    __tablename__ = "users"

    # 列定义 (对应 SQL 中的 CREATE TABLE)
    id = Column(Integer, primary_key=True, index=True)
    email = Column(String(255), unique=True, nullable=False)
    name = Column(String(100))
    hashed_password = Column(String(255))
    role = Column(Enum(UserRole), default=UserRole.MEMBER)
    created_at = Column(DateTime, default=datetime.utcnow)

    # 关系定义 (对应 SQL 中的 JOIN)
    topics = relationship("Topic", back_populates="creator")
    bindings = relationship("Binding", back_populates="user")

# 使用示例
# 查询所有管理员
admins = db.query(User).filter(User.role == UserRole.ADMIN).all()

```

```

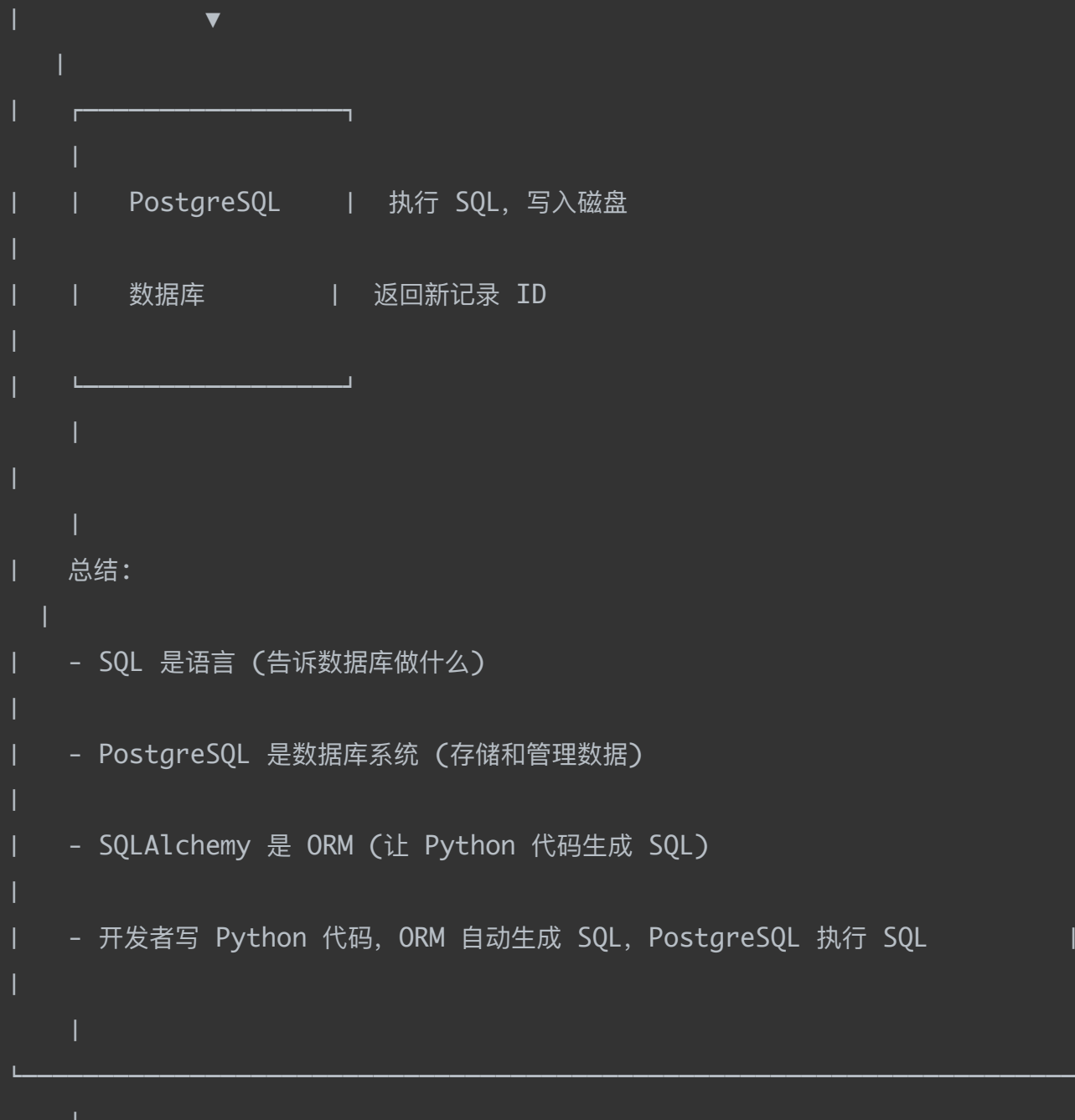
# 创建新用户
new_user = User(email="new@example.com", name="新用户")
db.add(new_user)
db.commit()

# 关联查询：获取用户创建的所有课题
user = db.query(User).filter(User.id == 1).first()
for topic in user.topics: # 自动执行 JOIN 查询
    print(topic.title)

```

3.5 数据库在项目中的位置





4. 后端框架之争: FastAPI vs Java Spring Boot

Java Spring 是企业级开发的老牌框架, FastAPI 是 Python 生态的新星。它们各有优势, 选择哪个取决于团队技术栈和项目需求。

4.1 代码对比: 实现同一个 API

让我们用实际代码来对比两者的开发体验。

需求：创建一个用户注册 API

- 接收用户名、邮箱、密码
- 验证邮箱格式
- 返回创建的用户信息

Spring Boot 实现 (Java)

```
// ===== 1. 实体类 (Entity) =====  
// src/main/java/com/tcgs/entity/User.java  
@Entity  
@Table(name = "users")  
@Data // Lombok 注解, 自动生成 getter/setter  
public class User {  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
  
    @Column(nullable = false, unique = true)  
    private String email;  
  
    @Column(nullable = false)  
    private String username;  
  
    @Column(nullable = false)  
    private String password;  
  
    @Column(name = "created_at")  
    private LocalDateTime createdAt = LocalDateTime.now();  
}  
  
// ===== 2. DTO (数据传输对象) =====  
// src/main/java/com/tcgs/dto/UserCreateRequest.java  
@Data  
public class UserCreateRequest {
```

```

    @NotBlank(message = "用户名不能为空")
    private String username;

    @NotBlank(message = "邮箱不能为空")
    @Email(message = "邮箱格式不正确")
    private String email;

    @NotBlank(message = "密码不能为空")
    @Size(min = 6, message = "密码至少6位")
    private String password;
}

// src/main/java/com/tcgs/dto/UserResponse.java
@Data
public class UserResponse {
    private Long id;
    private String username;
    private String email;
    private LocalDateTime createdAt;
}

// ===== 3. Repository (数据访问层) =====
// src/main/java/com/tcgs/repository/UserRepository.java
@Repository
public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByEmail(String email);
    boolean existsByEmail(String email);
}

// ===== 4. Service (业务逻辑层) =====
// src/main/java/com/tcgs/service/UserService.java
@Service
@RequiredArgsConstructor
public class UserService {
    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;
}

```

```

    public UserResponse createUser(UserCreateRequest request) {
        if (userRepository.existsByEmail(request.getEmail())) {
            throw new BusinessException("邮箱已被注册");
        }

        User user = new User();
        user.setUsername(request.getUsername());
        user.setEmail(request.getEmail());

        user.setPassword(passwordEncoder.encode(request.getPassword()));

        User saved = userRepository.save(user);
        return mapToResponse(saved);
    }

    private UserResponse mapToResponse(User user) {
        UserResponse response = new UserResponse();
        response.setId(user.getId());
        response.setUsername(user.getUsername());
        response.setEmail(user.getEmail());
        response.setCreatedAt(user.getCreatedAt());
        return response;
    }
}

```

// ===== 5. Controller (控制层) =====

// src/main/java/com/tcgs/controller/UserController.java

@RestController

@RequestMapping("/api/users")

@RequiredArgsConstructor

public class UserController {

private final UserService userService;

@PostMapping

public ResponseEntity<UserResponse> createUser(

```

        @Valid @RequestBody UserCreateRequest request) {
            UserResponse response = userService.createUser(request);
            return
ResponseEntity.status(HttpStatus.CREATED).body(response);
        }
    }
}

```

// 文件数量：5个文件，约 100+ 行代码

FastAPI 实现 (Python)

```

# ===== 1. 模型 + Schema 合并 =====
# app/models/user.py
from sqlalchemy import Column, Integer, String, DateTime
from datetime import datetime

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    email = Column(String(255), unique=True, nullable=False)
    username = Column(String(100), nullable=False)
    hashed_password = Column(String(255), nullable=False)
    created_at = Column(DateTime, default=datetime.utcnow)

# app/schemas/user.py
from pydantic import BaseModel, EmailStr
from datetime import datetime

class UserCreate(BaseModel):
    username: str
    email: EmailStr # 自动验证邮箱格式
    password: str

class UserResponse(BaseModel):
    id: int

```

```

    username: str
    email: str
    created_at: datetime

class Config:
    from_attributes = True # 支持从 ORM 对象转换

# ===== 2. 路由 (Controller + Service 合并)
=====
# app/api/users.py
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session
from passlib.context import CryptContext

router = APIRouter(prefix="/api/users", tags=["users"])
pwd_context = CryptContext(schemes=["bcrypt"])

@router.post("", response_model=UserResponse, status_code=201)
def create_user(
    request: UserCreate,
    db: Session = Depends(get_db)
):
    # 检查邮箱是否已注册
    if db.query(User).filter(User.email == request.email).first():
        raise HTTPException(status_code=400, detail="邮箱已被注册")

    # 创建用户
    user = User(
        username=request.username,
        email=request.email,
        hashed_password=pwd_context.hash(request.password)
    )
    db.add(user)
    db.commit()
    db.refresh(user)

```

```
return user # Pydantic 自动转换为 JSON
```

```
# 文件数量：2个文件，约 50 行代码
```

4.2 详细特性对比

FastAPI vs Spring Boot 详细对比			
特性	FastAPI (Python)	Spring Boot (Java)	
代码量	少 (约50行)	多 (约100+行)	
文件数	2-3个	5-6个	
启动时间	<1秒	5-15秒	
内存占用	~50MB	~200-500MB	
API 文档	自动生成 (内置)	需要 Swagger 配置	
类型检查	Pydantic 运行时	编译时	
异步支持	原生 async/await	WebFlux (复杂)	
学习曲线	平缓	陡峭	
企业级特性	需要额外配置	开箱即用	

	生态成熟度	年轻但活跃	非常成熟	
	招聘市场	数据/AI团队多	企业开发团队多	

4.3 Spring Boot 的优势场景

	Spring Boot 适合的场景	
	1. 大型企业级应用	
	- 复杂的事务管理（分布式事务）	
	- 完善的安全框架（Spring Security）	
	- 成熟的微服务生态（Spring Cloud）	
	2. 团队主要是 Java 背景	

- Java 工程师更容易上手
- IDE 支持完善 (IntelliJ IDEA)
- 编译时类型检查, 大型项目更安全

3. 需要与 Java 生态集成

-
- 对接 Kafka、Elasticsearch 等 Java 生态组件
 - 使用 Hibernate 复杂 ORM 特性
 - 需要 JVM 调优经验的高并发场景

代码示例 - Spring Security 配置:

```
@Configuration
@EnableWebSecurity

public class SecurityConfig {

    @Bean

    public SecurityFilterChain filterChain(HttpSecurity http)
        throws Exception {
```

4.4 FastAPI 的优势场景

- 团队主要是 Python 背景（算法工程师）

- 需要调用 NumPy、Pandas、PyTorch 等库

- 快速原型开发，迭代速度快

2. 中小型 API 服务

- 代码简洁，维护成本低

- 自动生成 API 文档，前后端协作方便

- 部署资源占用少

3. 高性能异步场景

- 原生 `async/await`，无需额外配置

- 基于 Starlette，性能接近 Go

- 适合 I/O 密集型应用（大量数据库/API调用）

代码示例 - FastAPI 异步 + 自动文档：


```

| | - **limit**: 每页返回数量, 1-100 |
|
| | """
| |
| | # 这些注释和类型提示会自动生成 Swagger 文档! | |
| | return await topic_service.list(status=status,
limit=limit)| |
|
|_____| |
|
| |
| 访问 http://localhost:8000/docs 自动看到交互式文档:
|
|
|_____| |
| | |_____| |
|
| | | GET /api/topics 获取课题列表 [Try it] |
| |
| | |_____| |
|
| | | Parameters: | |
| | |
| | | └─ status (query): 筛选状态 | |
|
| | | └─ limit (query): 每页数量 (default: 10) | |
|
| | |_____| |
|
|
|_____| |
|
| |
|_____|

```

4.5 TCGS 项目选择 FastAPI 的理由

为什么 TCGS 选择 FastAPI

1. 团队背景：算法团队，Python 是主力语言

→ 无需学习新语言，快速上手

2. 项目规模：中型管理系统，非高并发场景

→ FastAPI 性能完全够用

3. 开发效率：需要快速迭代，验证业务需求

→ Python 开发效率高，代码量少

4. API 文档：前后端协作频繁

→ Swagger 自动生成，减少沟通成本

5. 容器化部署：Docker + Docker Compose

→ Python 镜像小，启动快，资源占用少

如果是以下情况，会考虑 Spring Boot:

- 团队主要是 Java 背景
- 需要复杂的企业级特性（分布式事务、RBAC）
- 项目需要长期维护 10+ 年
- 公司技术栈统一为 Java

5. 数据库迁移：Alembic 的前世今生

数据库迁移是一个经常被忽视但极其重要的话题。这一章我们详细讲解：没有 Alembic 之前有多痛苦，有了 Alembic 之后工作流程如何改变。

5.1 没有 Alembic 的黑暗时代

场景：给 users 表添加 avatar 字段

没有迁移工具时的"解决方案":

传统方式的各种问题

方案 1: 手动维护 SQL 文件

项目根目录下创建:

/sql/

├─ 001_create_users.sql

├─ 002_add_avatar.sql ← 容易忘记

├─ 003_add_topics.sql

└─ 004_rename_column.sql ← 顺序搞错了怎么办?

问题:

- 哪些 SQL 已经执行过? 哪些没有? 没人知道
- 多人开发时, SQL 文件命名冲突

- 回滚怎么办？每个 SQL 都要写对应的 rollback

- 不同环境（开发/测试/生产）状态不一致

方案 2：每次部署前 DROP 所有表，重新创建

```
Base.metadata.drop_all(engine)
```

```
Base.metadata.create_all(engine)
```

问题：

- 生产环境敢这么干？数据全没了！

- 只适合开发环境，完全不可用于生产

方案 3：让 DBA 手动操作

开发："张哥，帮我在生产库加个字段"

DBA："行，发我 SQL"

开发："ALTER TABLE users ADD COLUMN avatar VARCHAR(500);"

5.2 Alembic 是什么?

步骤 1: 修改代码中的模型

app/models/user.py

class User(Base):

id = Column(Integer, primary_key=True)

email = Column(String(255))

avatar = Column(String(500)) # ← 新增这一行

步骤 2: 生成迁移文件 (Alembic 自动对比差异)

\$ alembic revision --autogenerate -m "add user avatar"

Alembic 做了什么:

1. 读取代码中的 SQLAlchemy 模型 (User.avatar)

2. 连接数据库, 读取当前表结构 (没有 avatar 列)

3. 对比差异: 代码有 avatar, 数据库没有


```
| | 4. 更新 alembic_version 表为最新版本 (a1b2c3d4) |
|
|
|_____|
|
| |
| 实际执行的 SQL:
|
| ALTER TABLE users ADD COLUMN avatar VARCHAR(500);
|
| UPDATE alembic_version SET version_num = 'a1b2c3d4';
|
|
|_____|
|_____|
```

5.4 迁移版本链 (类似 Git 提交历史)

```
|_____|
|_____|
|
|               迁移版本链示意
|
|_____|
|_____|
|
| |
| alembic/versions/ 目录下的文件:
|
| _____
|
| |
| |— 001_initial_tables.py          # 创建初始表
|
| |
| |— 002_add_user_avatar.py        # 添加头像字段
|
|
```

```

|   └─ 003_add_topics_table.py          # 添加课题表
|
|   └─ 004_add_wiki_module.py           # 添加 Wiki 模块
|
|   └─ 005_add_capacity_binding.py      # 添加产能绑定
|
|
|
|   版本链关系（每个文件记录自己的 down_revision):
|   ───────────────────────────────────
|
|
|   |
|   |
|   ▼
|   |
|   ───────────────────────────────────
|   | 001_initial_tables | revision = 'abc123'
|   |
|   | down_revision = None | ← 第一个版本, 没有上级
|   |
|   ───────────────────────────────────
|   |
|   |
|   |
|   ▼
|   |
|   ───────────────────────────────────
|   | 002_add_user_avatar | revision = 'def456'
|   |
|

```



```

| _____
|
| |
| |_____|
| |
| | version_num |
| |
| |_____|
| |
| | ghi789 | ← 当前数据库在这个版本
| |
| |_____|
| |
| |
| |
| 常用命令:
|
| _____
| |
| alembic upgrade head # 升级到最新版本
|
| alembic upgrade +1 # 升级一个版本
|
| alembic downgrade -1 # 回滚一个版本
|
| alembic downgrade base # 回滚到初始状态
|
| alembic history # 查看版本历史
|
| alembic current # 查看当前版本
|
|
|
|_____
|_____

```

5.5 Alembic 解决的核心问题

有了 Alembic 之后

问题 1: "我本地能跑, 测试环境报错"

之前: 开发者手动在本地执行 SQL, 忘记告诉别人

现在: 迁移文件随代码一起提交, 任何环境执行 `alembic upgrade head`

问题 2: "这个字段是谁加的? 什么时候加的?"

之前: 没人知道, 只能看数据库

现在: 每个迁移文件都有作者、日期、描述, 可追溯

问题 3: "上线出 bug 了, 需要回滚数据库"

之前: DBA 手动写回滚 SQL, 容易出错

现在: `alembic downgrade -1`, 自动执行 `downgrade()` 方法

问题 4: "多人开发, 数据库结构冲突"

之前: 两个人同时改表结构, 部署时互相覆盖

现在：Git 合并时发现迁移文件冲突，强制解决后再部署

问题 5： "新人入职，如何搭建开发环境"

之前：找 DBA 要最新的数据库备份，或者手动执行 N 条 SQL

现在：alembic upgrade head, 几秒钟自动同步到最新结构

TCGS 项目的实际应用：

docker-compose.yml 中的启动命令

backend:

command: >

sh -c "alembic upgrade head && # 先同步数据库结构

python seed.py && # 再插入初始数据

uvicorn app.main:app ..." # 最后启动服务

每次部署自动执行 alembic upgrade head, 确保数据库结构最新!

5.6 Alembic 迁移实战示例

示例：TCGS 项目中添加 Wiki 点赞功能的迁移

1. 首先在代码中添加模型

app/models/wiki.py

```
class WikiLike(Base):
```

```
    __tablename__ = "wiki_likes"
```

```
    id = Column(Integer, primary_key=True)
```

```
    page_id = Column(Integer, ForeignKey("wiki_pages.id"),
nullable=False)
```

```
    user_id = Column(Integer, ForeignKey("users.id"), nullable=False)
```

```
    created_at = Column(DateTime, default=datetime.utcnow)
```

联合唯一约束：一个用户只能给一篇文章点赞一次

```
    __table_args__ = (
```

```
        UniqueConstraint('page_id', 'user_id', name='uq_wiki_like'),
```

```
)
```

2. 生成迁移文件

```
# $ alembic revision --autogenerate -m "add wiki likes table"
```

3. 自动生成的迁移文件（可以手动调整）

```
# alembic/versions/xxx_add_wiki_likes_table.py
```

```
def upgrade():
```

```
    op.create_table(
```

```
        'wiki_likes',
```

```
        sa.Column('id', sa.Integer(), nullable=False),
```

```
        sa.Column('page_id', sa.Integer(), nullable=False),
```

```
        sa.Column('user_id', sa.Integer(), nullable=False),
```

```
        sa.Column('created_at', sa.DateTime(), nullable=True),
```

6. 前端架构：Vue 3 与 Pinia 状态管理

很多人混淆 Vue 和 Pinia 的关系，认为它们是同一个东西。这一章我们彻底搞清楚：**Vue 是 UI 框架，Pinia 是状态管理库**，它们解决的是完全不同的问题。

6.1 Vue 是什么？解决什么问题？

-
1. 声明式渲染：用模板描述 UI，数据变化自动更新 DOM
 2. 组件化开发：把页面拆分成可复用的组件
 3. 响应式系统：数据变化自动触发视图更新

一个简单的 Vue 组件：

```
<template>
  <div>
    <h1>{{ title }}</h1>          <!-- 数据绑定 -->
    <p>计数: {{ count }}</p>
    <button @click="count++">+1</button>  <!-- 事件绑定 -->
  </div>
</template>

<script setup lang="ts">
  import { ref } from 'vue'

  const title = ref('Hello Vue')
```

```

|   |   const count = ref(0)  // 响应式数据
|
|   |   // count 变化时，页面自动更新，无需手动操作 DOM
|   |   </script>
|   |
|
|_____|
|
|   |
|   |
|   |   Vue 解决的问题：
|   |
|   |_____
|   |
|   |   传统 JS: document.getElementById('count').innerHTML = newValue
|   |
|   |   Vue:      count.value++  // 自动更新 DOM
|   |
|   |
|   |
|_____|
|_____

```

6.2 为什么需要 Pinia? Vue 不够吗?

```

|_____
|_____
|
|           组件间状态共享的问题
|
|_____
|_____
|
|   |
|   |
|   |   场景： 多个组件需要共享同一份数据
|   |
|   |_____
|   |
|

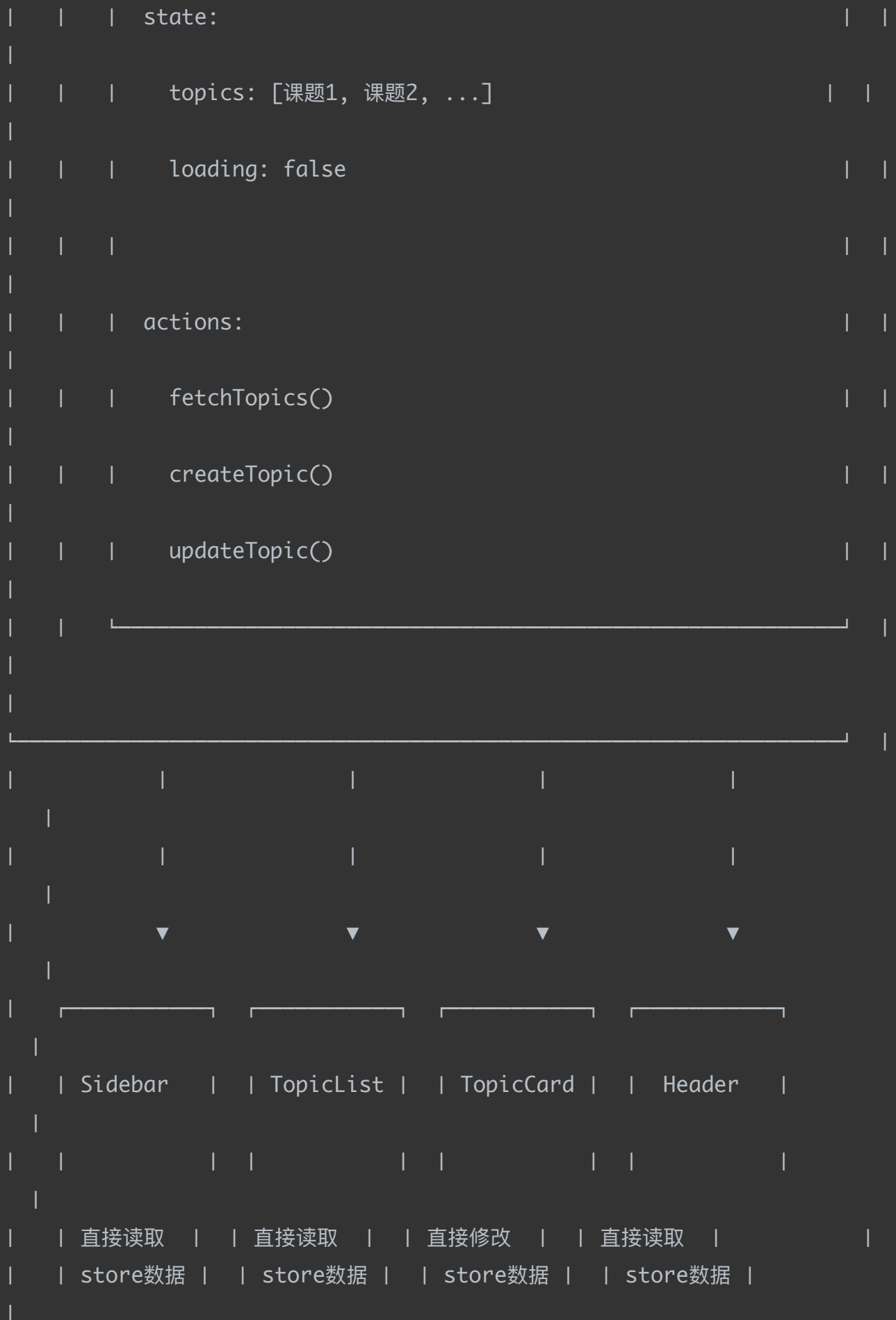
```



```
| 问题:
|
| - Props 层层传递, 代码臃肿 ("Prop Drilling")
|
| - 中间组件被迫接收和传递它不需要的数据
|
| - 数据更新要层层往上冒, 很容易出错
|
|
|_____
|_____
```

6.3 Pinia = 全局状态管理

```
|_____
|_____
|
| Pinia 解决方案
|
|_____
|_____
|
|
|
| 核心理念: 把共享数据放到一个"全局仓库"(Store)
|
| _____
|
|
|
|
|_____
| | Pinia Store (全局仓库) |
| |
| | _____
| |
| | | topicsStore |
| |
| | _____
| |
| |
```



优势：

- 任何组件都可以直接访问 Store，无需层层传递
- 数据修改后，所有使用该数据的组件自动更新
- 逻辑集中管理，易于维护和测试

6.4 Vue 和 Pinia 的区别总结

类比：

Vue = 房间里的家具（每个房间有自己的家具）
Pinia = 公共储物间（所有房间都可以存取东西）

什么时候需要 Pinia：

- ✓ 多个组件需要共享同一份数据（用户信息、课题列表）
- ✓ 数据需要跨页面保持（登录状态、购物车）
- ✓ 数据逻辑复杂，需要集中管理

什么时候不需要 Pinia：

- ✗ 数据只在一个组件内使用 → 用 `ref/reactive`
- ✗ 数据只在父子组件间传递 → 用 `props/emit`
- ✗ 简单的小应用 → 直接用 Vue 响应式就够了

6.5 TCGS 项目中的实际代码

Pinia Store 定义 (stores/topics.ts):

```
// stores/topics.ts
import { defineStore } from 'pinia'
import { ref, computed } from 'vue'
import { topicsApi } from '@/api/topics'
import type { Topic } from '@/types'

export const useTopicsStore = defineStore('topics', () => {
  // =====
  // State (状态) - 存储数据
  // =====
  const topics = ref<Topic[]>([])
  const loading = ref(false)
  const currentTopic = ref<Topic | null>(null)

  // =====
  // Getters (计算属性) - 派生数据
  // =====
  const openTopics = computed(() =>
    topics.value.filter(t => t.result === 'OPEN')
  )

  const topicCount = computed(() => topics.value.length)

  // =====
  // Actions (方法) - 修改数据、调用 API
  // =====
  async function fetchTopics() {
    loading.value = true
    try {
      const response = await topicsApi.list()
      topics.value = response.data // 更新状态, 所有组件自动更新
    } finally {
      loading.value = false
    }
  }
}
```

```

    }

    async function createTopic(data: TopicCreate) {
      const response = await topicsApi.create(data)
      topics.value.push(response.data) // 添加到列表, Sidebar 计数自动更新
      return response.data
    }

    async function updateTopic(id: number, data: Partial<Topic>) {
      const response = await topicsApi.update(id, data)
      const index = topics.value.findIndex(t => t.id === id)
      if (index !== -1) {
        topics.value[index] = response.data // 更新列表项
      }
      return response.data
    }

    // 返回需要暴露的内容
    return {
      topics,
      loading,
      currentTopic,
      openTopics,
      topicCount,
      fetchTopics,
      createTopic,
      updateTopic
    }
  })

```

组件中使用 Store:

```

<!-- Sidebar.vue - 显示课题数量 -->
<template>
  <div class="sidebar">
    <div class="stat">

```

```

        共 {{ topicsStore.topicCount }} 个课题
    </div>
    <div class="stat">
        进行中: {{ topicsStore.openTopics.length }}
    </div>
</div>
</template>

<script setup lang="ts">
import { useTopicsStore } from '@/stores/topics'

const topicsStore = useTopicsStore()
// 不需要 props, 直接访问全局 Store
// topicsStore.topicCount 变化时, 这里自动更新
</script>

```

```

<!-- TopicCard.vue - 编辑课题 -->
<template>
    <div class="topic-card">
        <h3>{{ topic.title }}</h3>
        <button @click="handleEdit">编辑</button>
    </div>
</template>

<script setup lang="ts">
import { useTopicsStore } from '@/stores/topics'

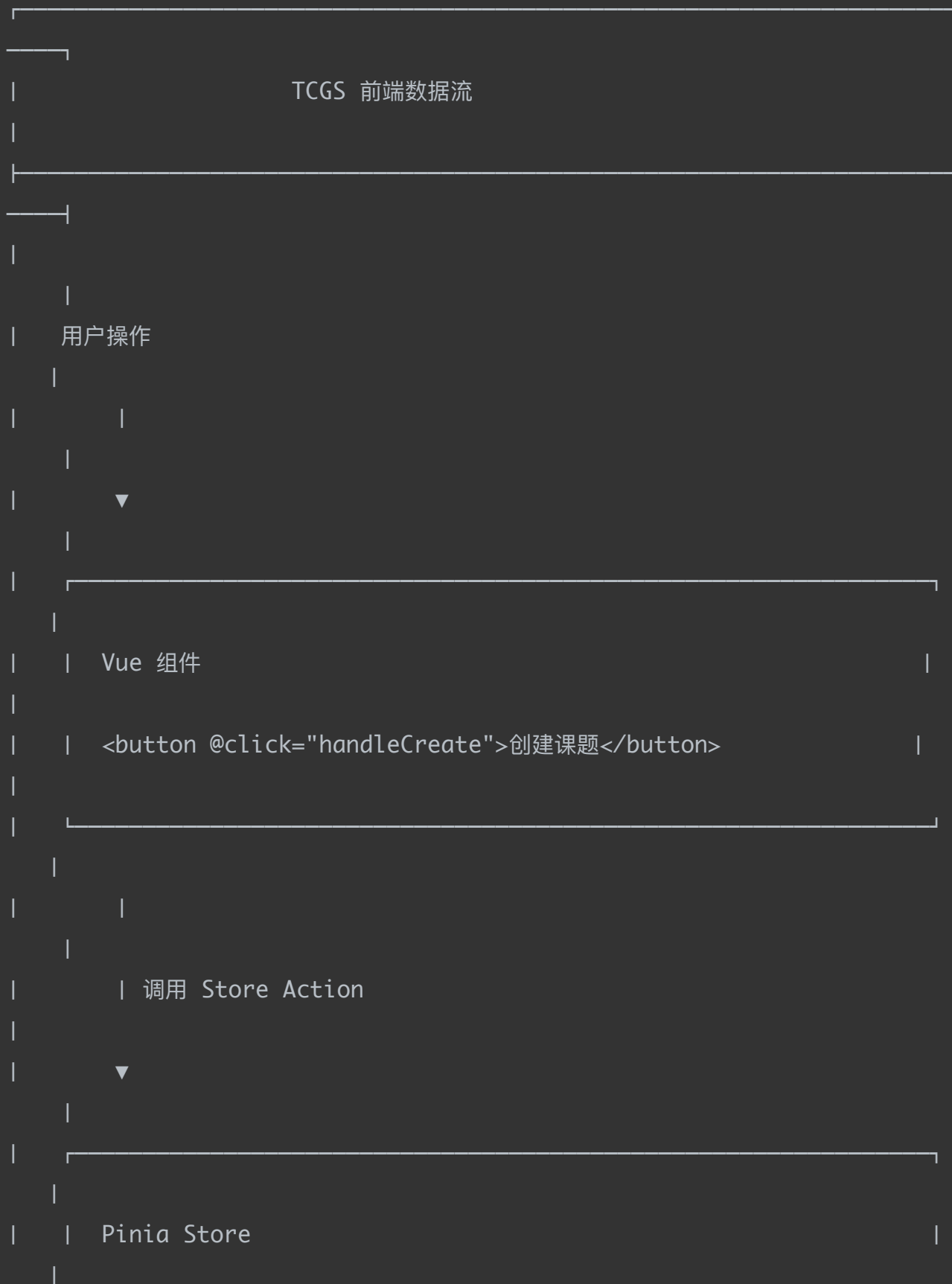
const props = defineProps<{ topic: Topic }>()
const topicsStore = useTopicsStore()

async function handleEdit() {
    await topicsStore.updateTopic(props.topic.id, { title: '新标题' })
    // 更新完成后, Sidebar 的计数、TopicList 的列表都会自动更新
    // 因为它们都在读取同一个 Store
}

```

</script>

6.6 数据流向总结



后端 (FastAPI + PostgreSQL)

响应返回



Pinia Store 更新状态

`topics.value.push(newTopic)`

Vue 响应式系统检测到变化

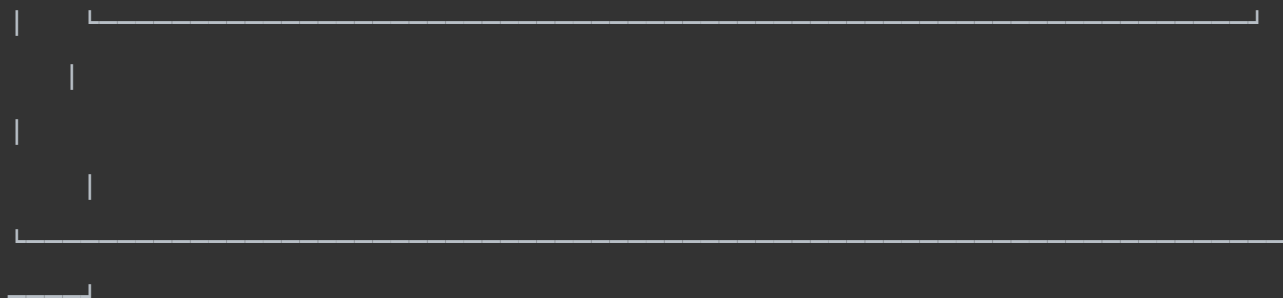


所有使用该数据的组件自动更新

- TopicList: 显示新课题

- Sidebar: 计数 +1

- Header: 显示最新动态



7. 容器化革命：Docker 如何解决依赖地狱

"在我机器上能跑啊！" —— 这可能是软件开发中最令人崩溃的一句话。Docker 的出现，彻底解决了这个问题。

7.1 Python 依赖地狱：没有 Docker 之前




```
| _____  
|  
| 开发者: macOS M1 芯片  
|  
| 服务器: Linux x86_64  
|  
| 某些 C 扩展包编译结果不兼容  
|  
|  
|  
|  
|_____  
_____
```

传统解决方案及其问题:

```
|_____  
|_____  
| 传统解决方案  
|  
|_____  
|_____  
|  
| 方案 1: virtualenv / venv  
|  
| _____  
|  
| python -m venv myenv  
|  
| source myenv/bin/activate  
|  
| pip install -r requirements.txt  
|  
|  
|  
|
```


- | 1. 安装 Python 3.11
- |
- | 2. 安装 PostgreSQL 开发库
- |
- | 3. 创建虚拟环境
- |
- | 4. 安装依赖
- |
- | 5. 配置环境变量
- |
- | 6. 启动服务
- |
- |
- |
- | 问题：步骤多、容易出错、每个环境都要手动操作
- |
- |

7.2 Docker 的解决方案：把一切都打包进去

- |
- |
- | Docker 的核心理念
- |
- |
- |
- |
- | Docker 镜像 = 完整的运行环境快照
- |
- |
- |
- |
- |
- |
- |


```

|   |   |_____|   |
|
|   |   |_____|   |
|
|   |   |   基础操作系统（精简版）   |   |   |
|   |   |   Debian slim / Alpine Linux   |   |
|
|   |   |_____|   |
|
|   |
|   |
|
|_____|   |
|
|   |
|   所有这些都打包在一个镜像里，任何地方运行都一样！   |
|
|   |
|_____|
_____

```

7.3 Dockerfile：打包的"菜谱"

TCGS 后端 Dockerfile 详解：

```

#
=====

=====
# 第 1 步：选择基础镜像（包含 Python 3.11 + Debian 精简版）
#
=====

=====
FROM python:3.11-slim
# 这一行就解决了 Python 版本问题！
# python:3.11-slim 包含：
#   - Debian Linux 精简版

```

```
# - Python 3.11 解释器
```

```
# - pip 包管理器
```

```
#
```

```
# 第 2 步：安装系统依赖
```

```
#
```

```
RUN apt-get update && apt-get install -y \  
    libpq-dev \      # PostgreSQL 客户端库 (psycopg2 需要)  
    gcc \            # C 编译器 (编译某些 Python 包)  
    curl \           # HTTP 工具  
    && rm -rf /var/lib/apt/lists/* # 清理缓存, 减小镜像体积
```

```
# 这一步解决了"libpq-dev 找不到"的问题!
```

```
#
```

```
# 第 3 步：安装 Python 依赖
```

```
#
```

```
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
# requirements.txt 包含:
```

```
# fastapi==0.115.0
```

```
# sqlalchemy==2.0.35
```

```
# psycopg2-binary==2.9.9
```

```
# uvicorn==0.30.6
```

```
# ...
```

```
# 这一步把所有 Python 依赖都装进镜像!
```

```
#
```

```
# 第 4 步：复制应用代码
```

```
#
```

```
COPY . .
```

```
# 把项目所有文件复制到镜像的 /app 目录
```

```
#
```

```
# 第 5 步：定义启动命令
```

```
#
```

```
EXPOSE 8000
```

```
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

构建和运行：

```
# 构建镜像（执行 Dockerfile 中的所有步骤）
```

```
docker build -t tcgs-backend .
```

```
# 运行容器（从镜像创建一个运行实例）
```

```
docker run -p 8000:8000 tcgs-backend
```

```
# 镜像可以推送到任何地方运行
```

```
docker push myregistry/tcgs-backend:v1.0
```

7.4 Docker 回答的关键问题

FAQ: Docker 与依赖

Q: Docker 能解决 Python 依赖问题吗?

A: 是的, 完全可以!

- Python 解释器: 打包在镜像里
- pip 包: 打包在镜像里
- 系统库: 打包在镜像里

镜像 = 完整的运行环境, 任何地方运行都一样

Q: 依赖可以打包进 Docker 镜像吗?

A: 是的! 这正是 Docker 的核心功能

构建时 (docker build):

```
FROM python:3.11-slim ← 基础环境
```

RUN apt-get install libpq-dev ← 系统依赖打包进去

RUN pip install -r requirements.txt ← Python 依赖打包进去

COPY . . ← 代码打包进去

运行时 (docker run):

容器直接使用镜像里的一切

不需要宿主机安装 Python

不需要宿主机安装任何依赖

只需要 Docker Engine

Q: 不同电脑上运行会有差异吗?

A: 不会! 这就是 Docker 的意义

开发者 A (macOS M1):

7.5 Docker Compose: 管理多个容器

TCGS 项目有多个服务，用 Docker Compose 一键启动：

```
# docker-compose.yml
version: '3.8'

services:

#
=====
# PostgreSQL 数据库
#
```

```
postgres:
  image: postgres:15-alpine      # 官方镜像，不用自己装数据库
  container_name: tcgs-postgres
  environment:
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: postgres
    POSTGRES_DB: tcgs
  volumes:
    - postgres_data:/var/lib/postgresql/data # 数据持久化
```

```
#
```

```
# FastAPI 后端
```

```
#
```

```
backend:
  build: ./backend      # 从 Dockerfile 构建
  container_name: tcgs-backend
  environment:
    DATABASE_URL: postgresql://postgres:postgres@postgres:5432/tcgs
    # ↑ postgres 是服务名，Docker 自动解析为容器 IP
  depends_on:
    - postgres          # 等 postgres 启动后再启动
  ports:
    - "8000:8000"
```

```
#
```

```
# Nginx + Vue 前端
```

```
#
```

```
frontend:
  build:
    context: .
    dockerfile: Dockerfile.frontend
  container_name: tcgs-frontend
```

```
    ports:
      - "80:80"
    depends_on:
      - backend

volumes:
  postgres_data: # 命名卷，数据不会随容器删除而丢失
```

一键启动整个项目：

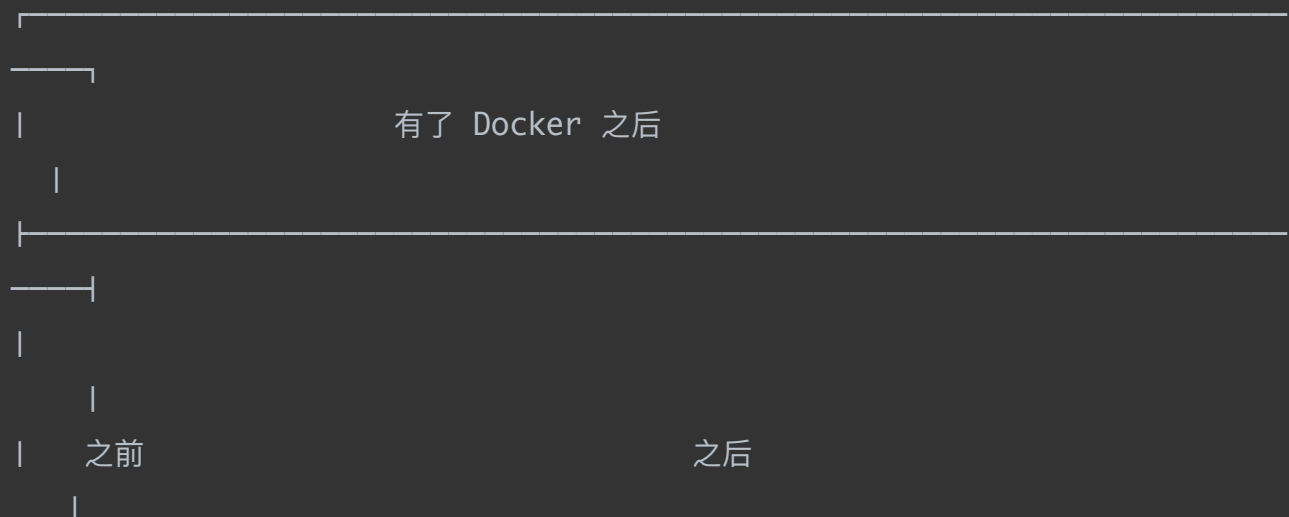
```
# 构建并启动所有服务
docker compose up -d --build

# 查看运行状态
docker compose ps

# 查看日志
docker compose logs -f backend

# 停止所有服务
docker compose down
```

7.6 Docker 带来的变革



1. 安装 Python 3.11

1. docker compose up

2. 安装 PostgreSQL

(完成)

3. 安装 libpq-dev

4. 创建虚拟环境

5. pip install

6. 配置环境变量

7. 创建数据库

8. 运行迁移

9. 启动服务

(可能还会遇到各种报错)

新人入职：

之前：花一天时间搭环境

之后：git clone → docker compose up → 5分钟开始开发

8. 反向代理：为什么需要 Nginx

很多人会问：前端和后端都能独立运行，为什么还要加一层 Nginx？直接访问不行吗？这一章我们来解答这个问题。

8.1 没有 Nginx 会怎样?

后端 API 服务: `http://localhost:8000` (FastAPI)

问题 1: 跨域错误 (CORS)

浏览器访问: `http://localhost:5173`

前端调用: `fetch('http://localhost:8000/api/topics')`

 报错: Access to fetch has been blocked by CORS policy

原因: 浏览器同源策略

- 源 = 协议 + 域名 + 端口

- `localhost:5173` $\neq$ `localhost:8000`

- 浏览器禁止跨源请求 (安全机制)

问题 2: 生产环境访问混乱

用户需要知道两个地址：

- 页面：http://example.com:80
- API：http://example.com:8000/api

前端代码需要硬编码后端地址：

```
const API_URL = 'http://example.com:8000' // 不同环境要改
```

问题 3：端口暴露，安全隐患

-
- 8000 端口对外暴露
 - 数据库端口 5432 可能也暴露
 - 攻击面增大

8.2 Nginx 反向代理如何解决这些问题



8.3 Nginx 配置详解

nginx.conf - TCGS 项目配置

```
server {  
    listen 80;                # 监听 80 端口  
    server_name _;           # 匹配所有域名
```

```
#
```

```
# 静态文件服务 + SPA 路由支持
```

```
#
```

```
root /usr/share/nginx/html;    # Vue 构建产物目录
index index.html;
```

```
location / {
    try_files $uri $uri/ /index.html;
    #
    # 这行配置非常关键！解释：
    # 1. 用户访问 /topics
    # 2. Nginx 先找 /topics 文件 → 不存在
    # 3. 再找 /topics/ 目录 → 不存在
    # 4. 最后返回 /index.html
    # 5. Vue Router 在前端处理 /topics 路由
    #
    # 如果没有这行，刷新页面会 404！
    # 因为 /topics 是前端路由，服务器上没有这个文件
}
```

```
#
```

```
# API 反向代理（核心！）
```

```
#
```

```
location /api/ {
    proxy_pass http://backend:8000;
    #
    # 请求转发示例：
    # 用户请求：GET http://example.com/api/topics
    # Nginx 转发：GET http://backend:8000/api/topics
    #
    # backend 是 Docker 服务名，Docker 内部 DNS 自动解析
}
```

```

    proxy_http_version 1.1;

    # 传递客户端真实信息给后端
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}

#
=====

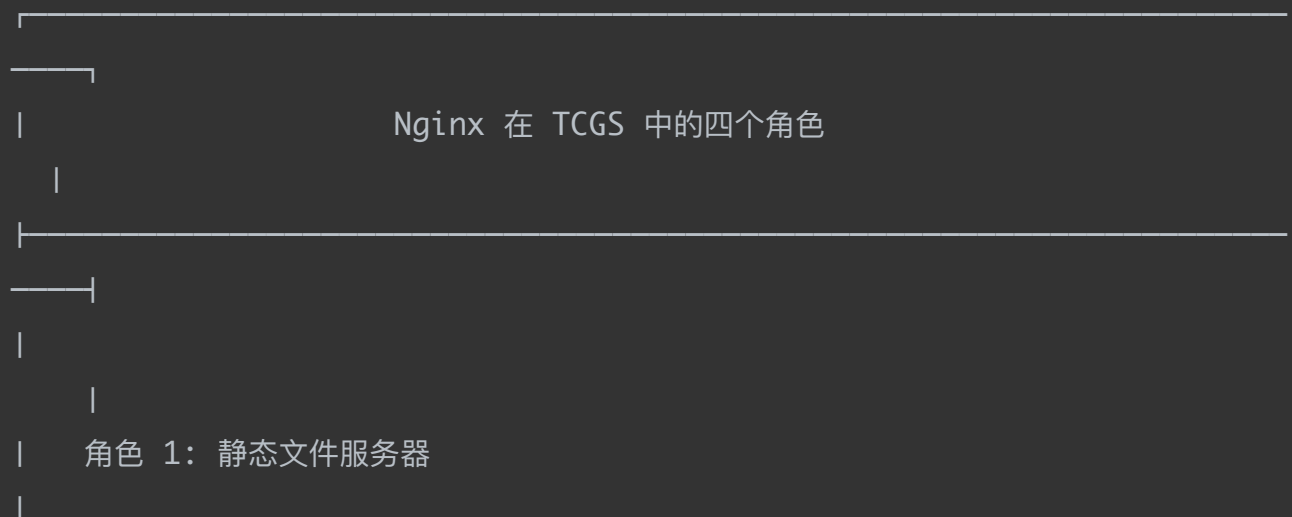
# FastAPI 自动文档
#
=====

location /docs {
    proxy_pass http://backend:8000/docs;
}

location /openapi.json {
    proxy_pass http://backend:8000/openapi.json;
}
}

```

8.4 Nginx 的多重角色



请求: /index.html, /assets/logo.png, /favicon.ico

处理: 直接从磁盘读取文件返回

优势: Nginx 处理静态文件比 Python 快 10 倍以上

角色 2: SPA 路由支持

请求: /topics, /capacity, /wiki/pages/123

处理: try_files 找不到 → 返回 index.html → Vue Router 处理

意义: 让前端路由在刷新后也能正常工作

角色 3: API 反向代理

请求: /api/*

处理: 转发到 backend:8000

意义: 解决跨域、隐藏后端服务

角色 4: 统一入口 (API 网关雏形)



8.5 请求流转完整示例



```
|         - 匹配 location / { try_files ... }  
|  
|         - 找 /topics 文件 → 不存在  
|  
|         - 返回 /index.html
```

```
|  
|         |  
|         |  
|         ▼  
|         |
```

| 步骤 3: 浏览器加载 index.html

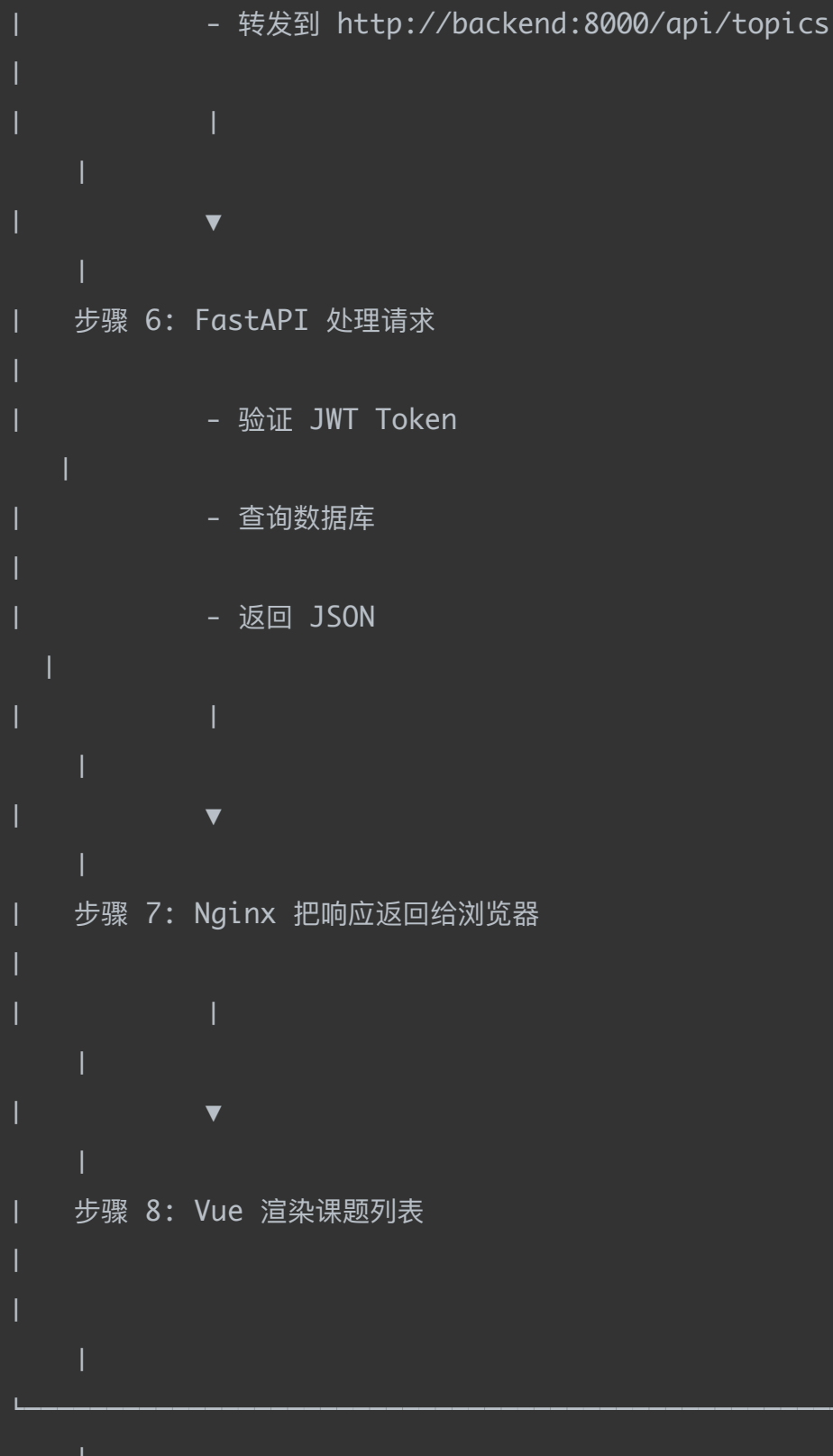
```
|  
|         - 下载 Vue 应用 JS/CSS  
|  
|         - Vue Router 解析 /topics 路由  
|  
|         - 渲染 TopicsListView 组件  
|  
|         |  
|         |  
|         ▼
```

| 步骤 4: Vue 组件发起 API 请求

```
|         fetch('/api/topics')  
|  
|         // 注意是相对路径，同源，无跨域问题!  
|         |  
|         |  
|         ▼
```

| 步骤 5: Nginx 收到 /api/topics 请求

```
|  
|         - 匹配 location /api/ { proxy_pass ... }
```



8.6 为什么不直接在 FastAPI 里配置 CORS?

方案 B: Nginx 反向代理 (生产环境推荐)

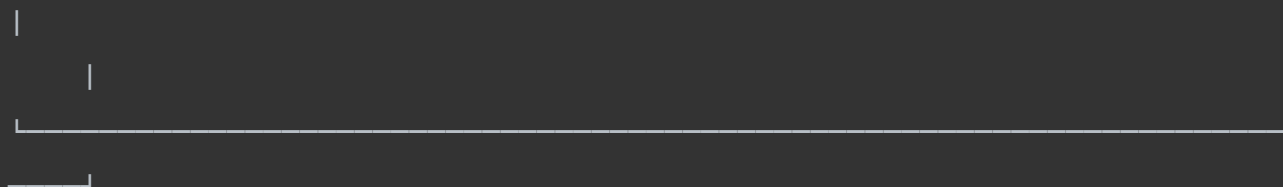
所有请求都通过 Nginx, 前端和 API 同源

优势:

- 根本不存在跨域问题 (同源)
- 无需维护白名单
- 后端服务不直接暴露, 更安全
- 静态文件服务更高效

TCGS 项目的选择:

- 开发环境: FastAPI CORS (方便 Vite 热重载)
- 生产环境: Nginx 代理 (安全、高效)



9. 对象存储：MinIO 是什么

TCGS 项目中有个 MinIO 服务，很多人不知道它是干什么的。这一章我们来解释：为什么需要对象存储，MinIO 解决什么问题。

9.1 传统文件存储的问题



| | └─ slides.pptx

|

| └─ ...

|

|

| 问题 1: 容器重启文件丢失

|

|

| Docker 容器是无状态的!

|

| docker compose down → 容器删除 → /app/uploads/ 目录消失

|

|

| 问题 2: 多实例无法共享

|

|

| 如果部署多个后端实例做负载均衡:

| - 用户 A 上传到 backend-1

|

| - 用户 B 请求到 backend-2 → 找不到文件!

|

|

| 问题 3: 备份和迁移困难

|

|

| - 文件分散在服务器各处

|

| - 需要手动同步到备份服务器

|

- 迁移时容易遗漏

问题 4：无法直接对外提供下载链接

文件在后端容器内，用户下载需要：

请求 → 后端读取文件 → 返回文件流

后端成为瓶颈，占用大量内存和带宽

9.2 对象存储是什么

对象存储 vs 文件系统

文件系统（传统）：

/home/user/documents/project/report.pdf

- 层级目录结构

- 文件有路径、权限、修改时间等属性

- 依赖操作系统
-
-
-
- 对象存储（现代）：
 - _____
 -
 - Bucket: tcgs-attachments
 -
 - Object Key: topics/1/report.pdf
 -
 - 扁平的键值存储 (Key → Object)
 -
 - 每个对象有元数据（大小、类型、自定义属性）
 - 通过 HTTP API 访问
 -
 - 天然支持分布式、高可用
-
-
-



Amazon S3 (Simple Storage Service):

- AWS 的对象存储服务
- 业界标准，几乎所有云厂商都兼容 S3 API
- 按存储量和流量收费

MinIO:

- 开源的 S3 兼容存储
- 可以自己部署，完全免费
- API 与 S3 100% 兼容
- 适合开发环境、私有云、边缘计算

类比:

MySQL (商业版) ↔ MariaDB (开源替代)

Amazon S3 (云服务) ↔ MinIO (开源替代)

TCGS 项目选择 MinIO 的理由：

✓ 开发环境免费，不需要 AWS 账号

✓ Docker 一键启动

✓ 与 S3 API 兼容，生产环境可无缝切换到 AWS S3

✓ 自带 Web 管理界面（端口 9001）

9.4 MinIO 在 TCGS 中的使用

文件上传流程

用户上传附件

问题 1: 容器重启文件丢失

解决: MinIO 数据存储在 Docker Volume, 独立于容器

`docker compose down` → 容器删除 → `minio_data` 保留

问题 2: 多实例无法共享

解决: 所有后端实例都连接同一个 MinIO

`backend-1` 上传 → MinIO → `backend-2` 可以读取

问题 3: 备份和迁移困难

解决: MinIO 提供标准 API, 可以用 `mc` (MinIO Client) 工具同步

`mc mirror minio/tcgs-attachments backup/`

问题 4: 后端成为文件下载瓶颈

解决: MinIO 支持预签名 URL, 用户直接从 MinIO 下载

生成临时下载链接 (有效期 1 小时)

`url = minio_client.presigned_get_object(`

`bucket="tcgs-attachments",`

`object_key="topics/1/report.pdf",`

`expires=timedelta(hours=1)`

```
|      )  
|  
|      # 返回: http://minio:9000/tcgs-  
attachments/topics/1/report.pdf?签名  
|      # 用户直接访问这个 URL 下载, 不经过后端
```

```
|  
|  
|
```

```
|  
|
```

```
| 生产环境迁移:
```

```
|  
|
```

```
| 开发环境: MinIO (本地 Docker)
```

```
|  
|
```

```
| 生产环境: AWS S3 / 阿里云 OSS / 腾讯云 COS
```

```
|  
|
```

```
| 因为 API 兼容, 只需要改配置:
```

```
|  
|
```

```
| # 开发环境
```

```
|  
|
```

```
| MINIO_ENDPOINT=minio:9000
```

```
|  
|
```

```
| MINIO_ACCESS_KEY=minioadmin
```

```
|  
|
```

```
| # 生产环境 (切换到 AWS S3)
```

```
|  
|
```

```
| S3_ENDPOINT=s3.amazonaws.com
```

```
|  
|
```

```
| S3_ACCESS_KEY=AKIA...
```

```
|
```

```
| # 代码不用改!
```

```
|
```

```
|
```

```
|
```

```
|_____
```

```
_____|
```

9.6 MinIO 管理界面

```
|_____
```

```
_____|
```

```
| MinIO Console (Web 管理界面)
```

```
|
```

```
|_____
```

```
_____|
```

```
|
```

```
|
```

```
| 访问: http://localhost:9001
```

```
|
```

```
| 账号: minioadmin
```

```
|
```

```
| 密码: minioadmin
```

```
|
```

```
|
```

```
|
```

```
| 功能:
```

```
|
```

```
| - 查看所有 Bucket
```

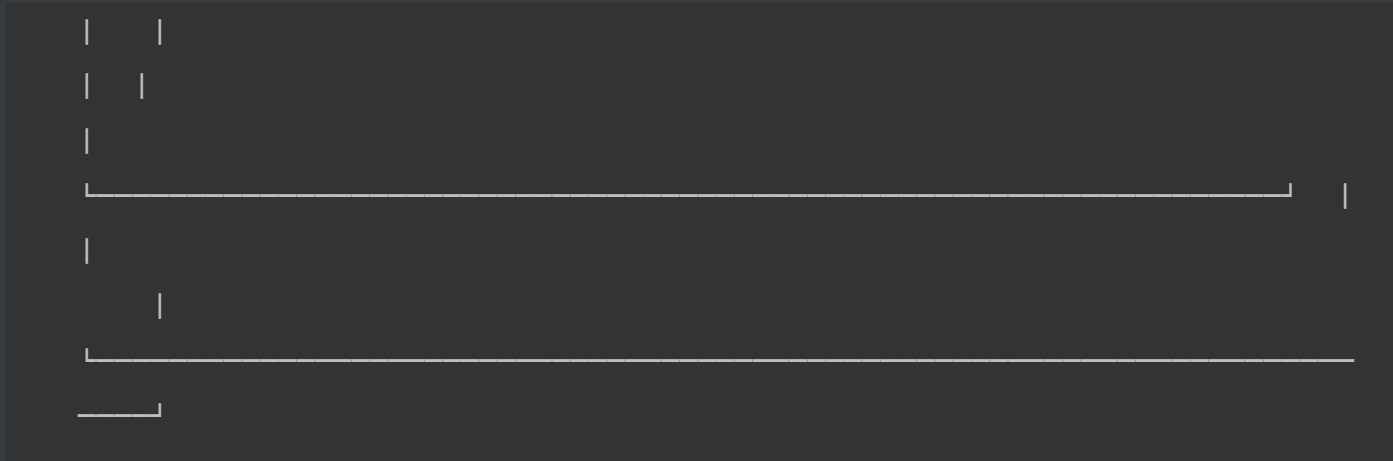
```
|
```

```
| - 浏览和下载文件
```

```
|
```

```
| - 上传文件 (用于测试)
```

```
|
```

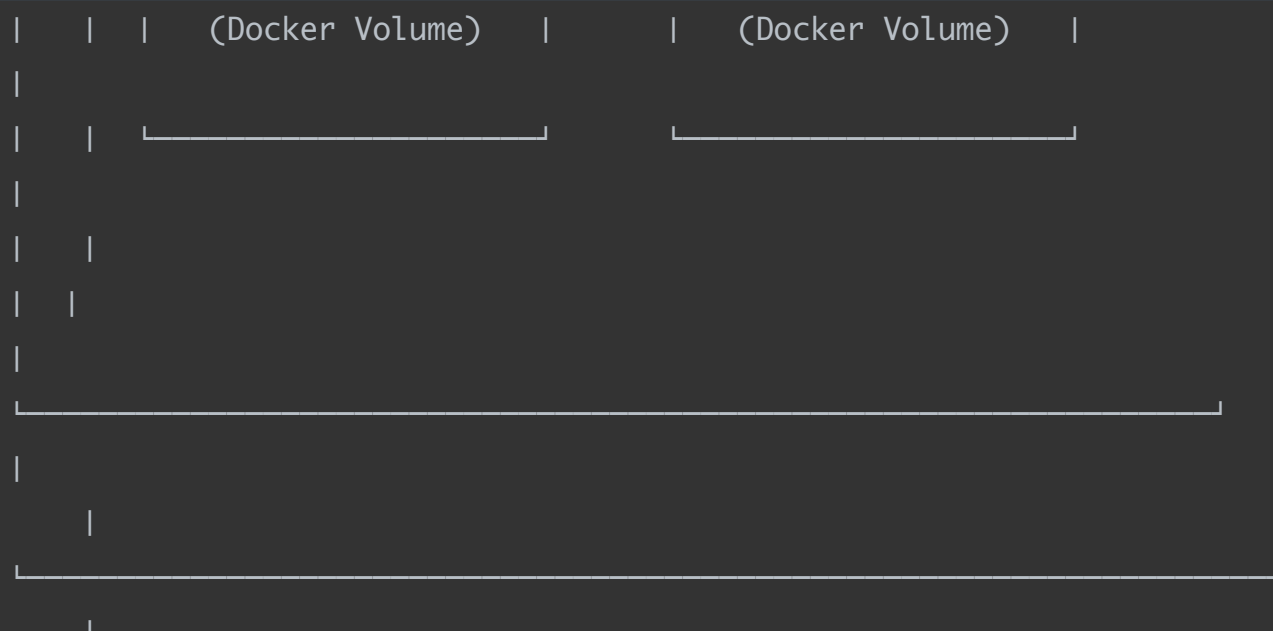



10. 系统架构图与请求流转

前面我们分别介绍了各个技术组件，这一章我们把它们串联起来，看看整个系统是如何协同工作的。

10.1 部署架构全景图





10.2 完整请求流转示例

场景：用户登录后创建一个课题



```
| |
| |
| ▼
| |
| 2 authStore.login({ email, password })
| |
| |
| ▼
| |
| 3 Axios: POST /api/auth/login
| |
| | { "email": "user@example.com", "password": "xxx" }
| |
| |
| ▼
| |
| 4 Nginx: proxy_pass → http://backend:8000/api/auth/login
| |
| |
| ▼
| |
| 5 FastAPI: 验证密码 → 生成 JWT Token
| |
| | response: { "access_token": "eyJhbG...", "user": {...} }
| |
| |
| ▼
| |
| 6 authStore: 保存 token 到 localStorage, 更新 user 状态
| |
| |
```


7 Vue Router: 跳转到 /dashboard

阶段 2：创建课题

8 用户在 TopicsListView.vue 点击"创建课题"

填写表单：标题、类型、紧急度等


```
9 topicsStore.createTopic(formData)
```

```
loading.value = true
```

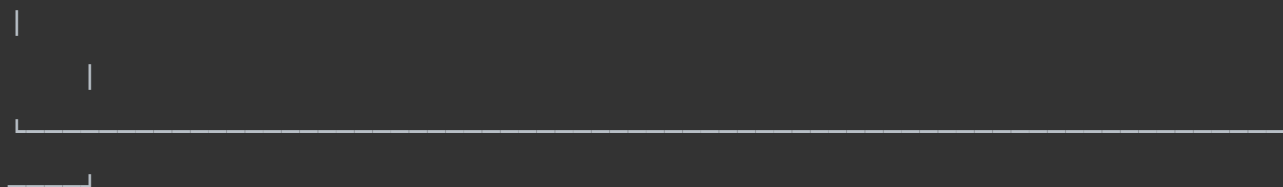

10 Axios: POST /api/topics

```
Headers: { Authorization: "Bearer eyJhbG..." }
```

```
Body: { "title": "新课题", "type": "RESEARCH", ... }
```



```
| | | | └─ topics.py      # /api/topics/*
|
| | | | └─ ...
|
| | | └─ models/          # SQLAlchemy ORM 模型
|
| | | | └─ user.py
|
| | | | └─ topic.py
|
| | | | └─ ...
|
| | | └─ schemas/         # Pydantic 数据验证
|
| | └─ services/          # 业务逻辑层
|
| └─ alembic/             # 数据库迁移
|
| | └─ versions/          # 迁移脚本
|
| └─ requirements.txt     # Python 依赖
|
| └─ Dockerfile           # 后端镜像构建
|
|
|
| ─────────────────── 部署配置 ───────────────────
|
| └─ docker-compose.yml   # 多容器编排
|
| └─ Dockerfile.frontend  # 前端镜像（多阶段构建）
|
| └─ nginx.conf           # Nginx 配置
|
| └─ .env                 # 环境变量
|
```



10.4 技术栈协同关系




```
| 1. TypeScript interface ↔ Pydantic BaseModel (类型一致性)
|
| 2. camelCase ↔ snake_case (Axios 拦截器自动转换)
|
| 3. Pinia 响应式 → Vue 自动更新 (无需手动操作 DOM)
| 4. SQLAlchemy ORM → SQL (无需手写 SQL)
|
|
|
|_____
|_____
```

11. Vue 响应式原理

Vue 3 的响应式系统是其核心特性，理解它有助于写出更高效的代码。

11.1 什么是响应式？

```
|_____
|_____
|                                     响应式的本质
|
|_____
|_____
|
|
|
| 传统方式（命令式）：
|
| _____
|
| let count = 0
|
| document.getElementById('counter').innerHTML = count
|
```




Virtual DOM Diff 与 key

没有 key 的情况（低效）：

旧列表：

`<li>课题A</li>`

`<li>课题B</li>`

→

新列表：

`<li>课题C</li>` ← 内容替换

`<li>课题A</li>` ← 内容替换

`<li>课题B</li>` ← 新增

执行了 3 次 DOM 操作

有 key 的情况（高效）：

旧列表：

`<li key="1">课题A</li>`

`<li key="2">课题B</li>`

→

新列表：

`<li key="3">课题C</li>` ← 只新增

`<li key="1">课题A</li>` ← 复用+移动

```

|                                     <li key="2">课题B</li>  ← 复用
|
|
|
|
|  Vue 通过 key 识别"这是同一个元素", 只做必要的 DOM 操作
|
|
|
|  最佳实践:
|
|  _____
|
|
|  <li v-for="topic in topics" :key="topic.id">  ← 使用唯一 ID
|
|      {{ topic.title }}
|
|  </li>
|
|
|
|_____
_____

```

12. 4+1 视图模型

12.1 逻辑视图 (Logical View)

```

|_____
|
|_____
|
|      逻辑视图 - 功能分解
|
|_____
|_____
|
|_____

```


	职责分配:	

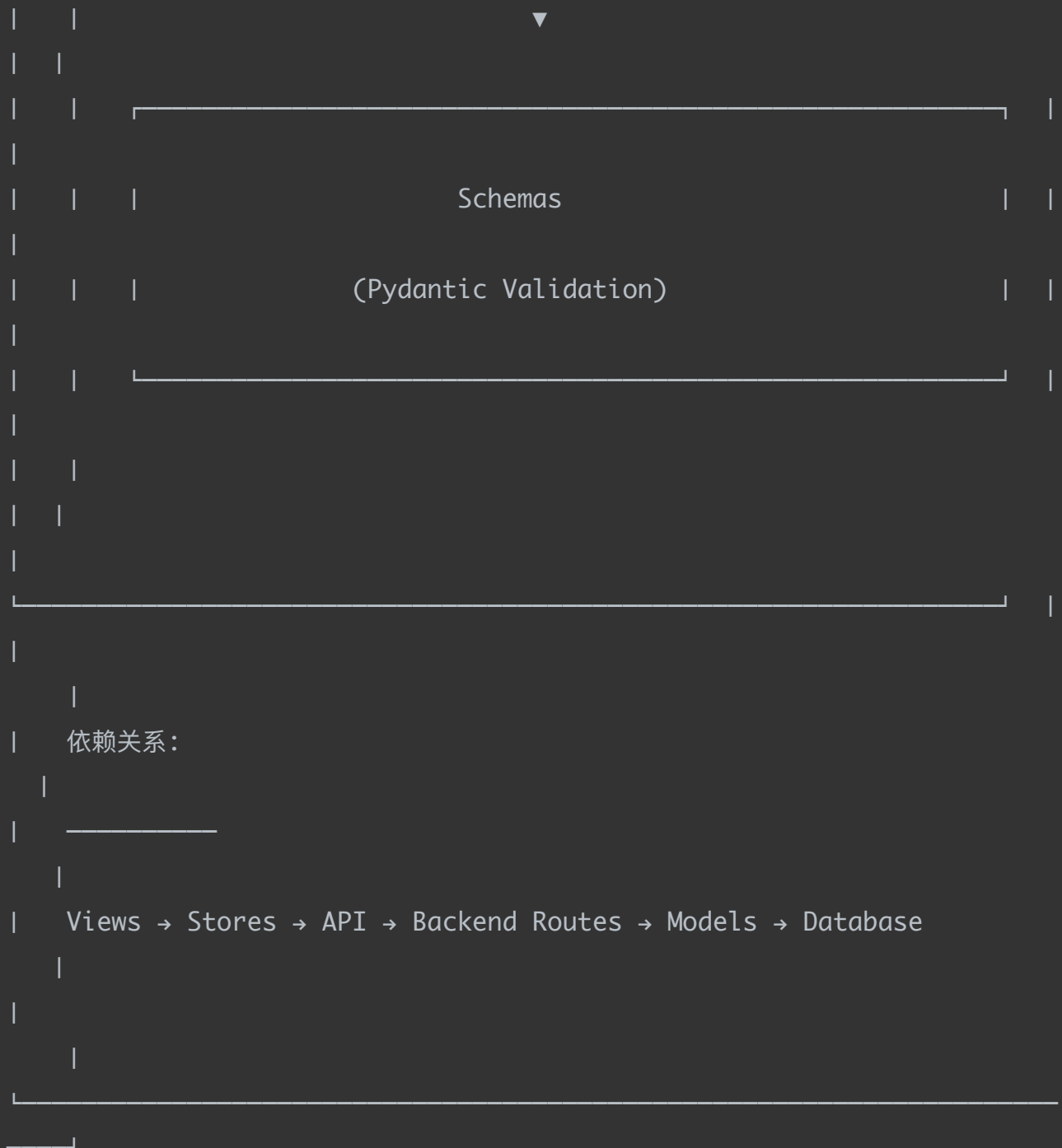
	Auth: 用户认证、授权、会话管理	
	Topics: 课题 CRUD、阶段流转、评审、交付物	
	Capacity: 槽位管理、人员绑定、工作量统计	
	Wiki: 文档管理、版本控制、评论互动	
	Insights: 数据聚合、KPI 计算、可视化	

12.2 开发视图 (Development View)

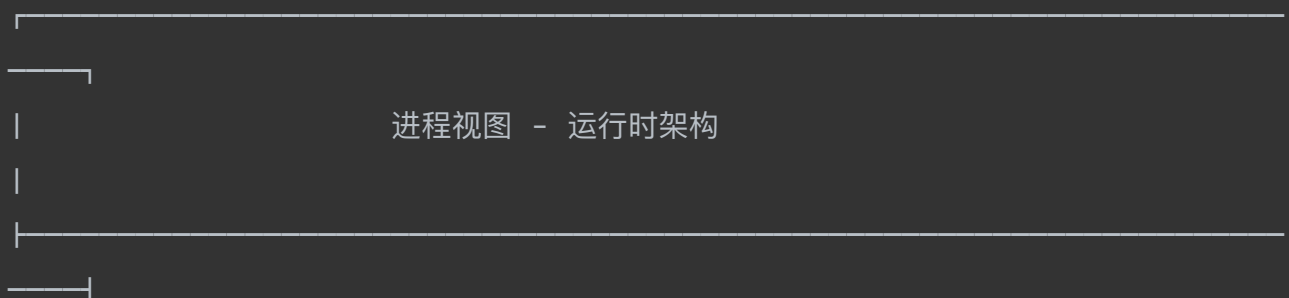
	开发视图 - 代码组织	

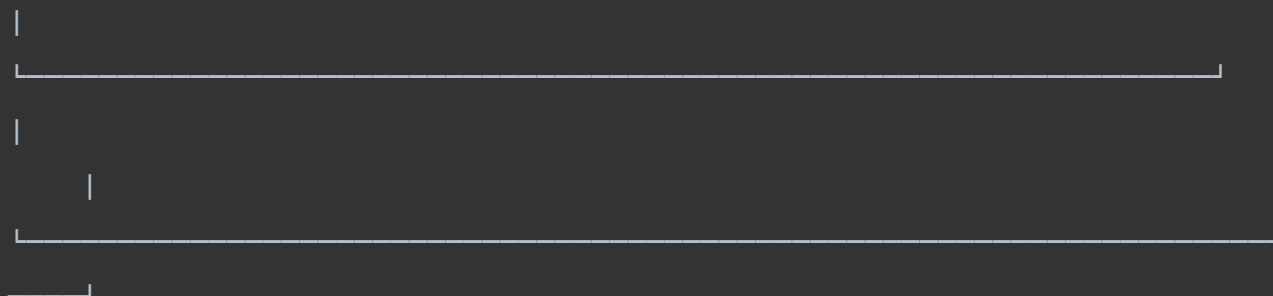
	前端包 (Frontend)	





12.3 进程视图 (Process View)





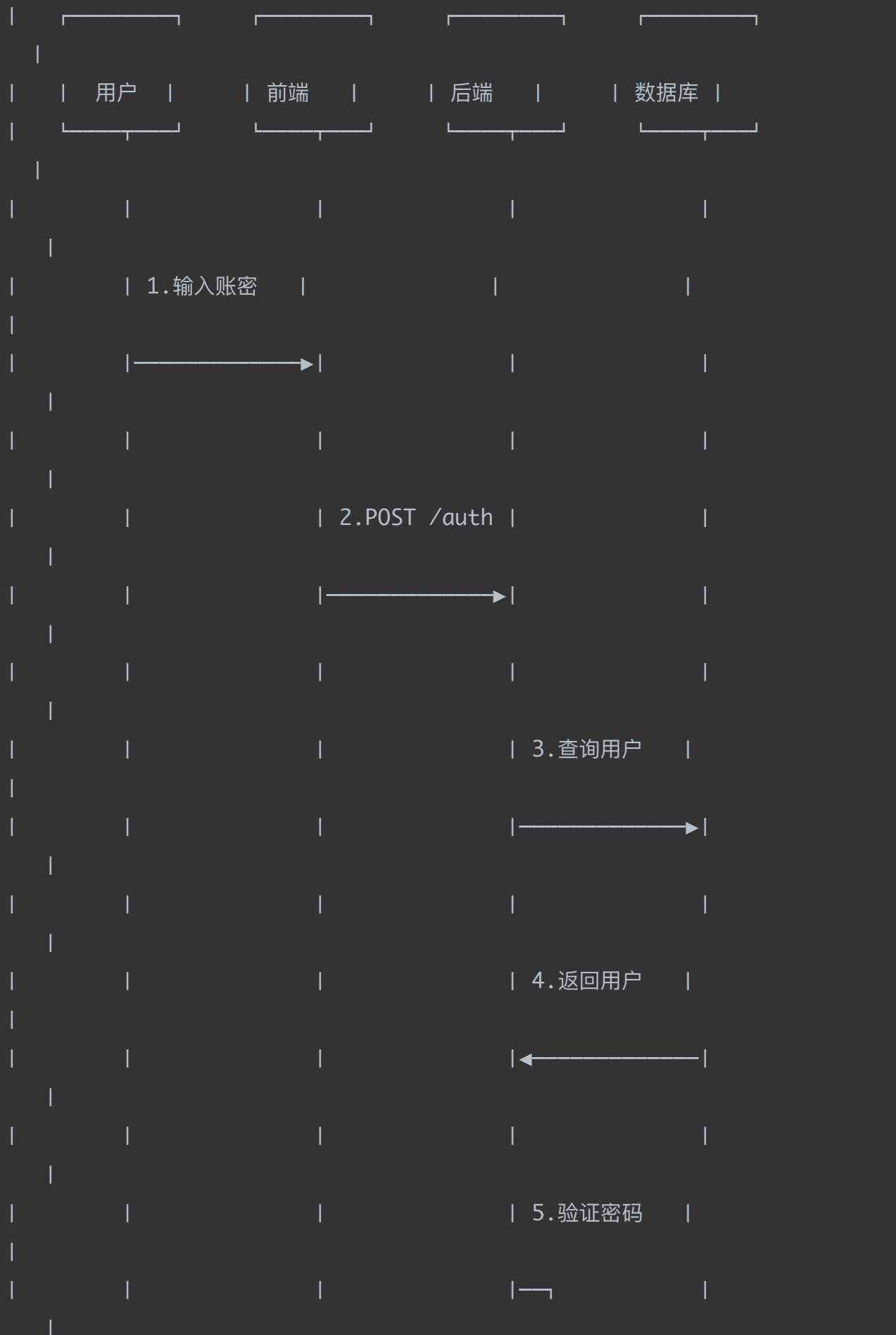
12.4 物理视图 (Physical View)



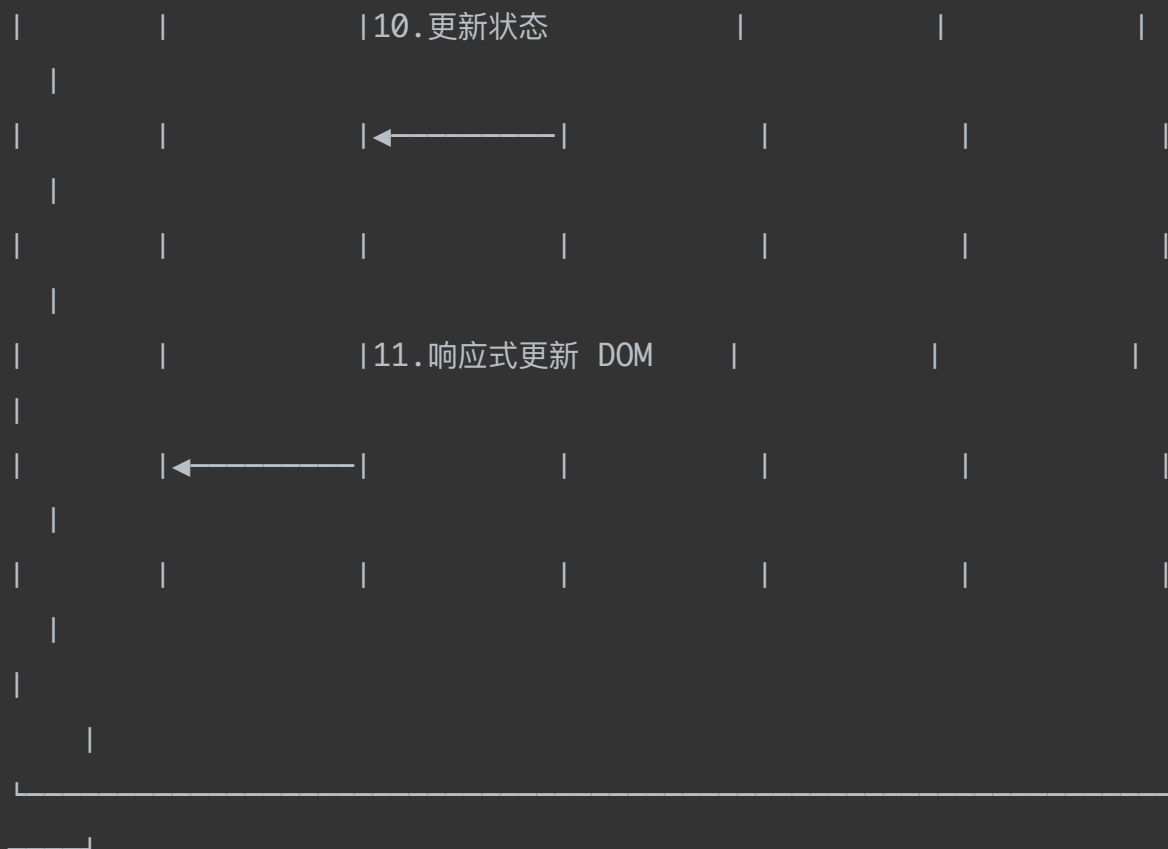


12.5 场景视图 (Scenarios)





		4.createTopic()			
		—————▶			
		5.POST			
		—————▶			
			6.验证JWT		
			—————▶		
				7.INSERT	
				—————▶	
				8.新记录	
				—————◀	
			9.课题数据		
			—————◀		



附录: 常用命令速查

```
# -----  
# Docker 操作  
# -----  
  
docker compose up -d --build    # 构建并启动所有服务  
docker compose down             # 停止所有服务  
docker compose down -v          # 停止并删除数据卷 (⚠ 数据丢失)  
docker compose logs -f backend  # 查看后端日志  
docker compose ps               # 查看服务状态  
docker exec -it tcgs-backend sh # 进入后端容器  
  
# -----  
# 数据库操作  
# -----  
  
docker exec -it tcgs-postgres psql -U postgres -d tcgs  
# 在 psql 中:
```

```
\dt                                # 列出所有表
\d topics                          # 查看表结构
SELECT * FROM users;              # 查询数据

# -----
# Alembic 迁移
# -----
alembic revision --autogenerate -m "add new column" # 生成迁移
alembic upgrade head                # 应用所有迁移
alembic downgrade -1               # 回滚一个版本
alembic history                    # 查看迁移历史

# -----
# 前端开发
# -----
npm run dev                        # 启动开发服务器
npm run build                      # 生产构建
npm run lint                       # 代码检查
```

报告完成于: 2026-04-02

作者: Claude Code

版本: 1.0